PCCST 501 COMPUTER NETWORKS

SEMESTER S5

COMPUTER NETWORKS

(Common to CS/CD/CM/CR/CA/AD/AI/CB/CN/CU/CI)

Course Code	**PCCST501**	**CIE Marks**	40
Teaching Hours/Week (L: T:P: R)	3:1:0:0	**ESE Marks**	60
Credits	4	**Exam Hours**	2 Hrs. 30 Min.
Prerequisites (if any)	None	**Course Type**	Theory

Course Objectives:

1. To introduce the core concepts of computer networking.
2. To develop a big picture of the internetworking implementation on Linux-based systems.
3. To impart an overview of network management concepts.

MODULE I

Module No.	Syllabus Description	Contact Hours
1	Overview of the Internet, Protocol layering (Book 1 Ch 1) Application Layer: Application-Layer Paradigms, Client-server applications - World Wide Web and HTTP, FTP. Electronic Mail, DNS. Peer-to-peer paradigm - P2P Networks, Case study: BitTorrent (Book 1 Ch 2)	6

Introduction to Computer Networks

What is a Computer Network?

A **computer network** is a collection of interconnected devices that can communicate and share data with each other.

Simple Definition

A computer network is the interconnection of devices capable of exchanging information.

These devices may include:

- Desktop computers
- Laptops
- Smartphones
- Servers
- Printers
- Routers
- Switches

The connection between devices can be established using:

- **Wired media** → cables such as Ethernet or fiber optics
- **Wireless media** → Wi-Fi, Bluetooth, cellular communication, etc.

Why Do We Need Computer Networks?

1. Resource Sharing

Computer networks allow multiple users to share hardware and software resources efficiently.

- Printers
- Files and folders
- Internet connection
- Software applications

2. Communication

Networks enable fast and effective communication among users across different locations.

- Email
- Video conferencing
- Instant messaging
- Voice and video calls

3. Data Sharing

Networks facilitate easy access, storage, and exchange of information.

- Databases
- Cloud storage
- Multimedia content (images, audio, video)
- Shared documents

4. Improved Reliability and Efficiency

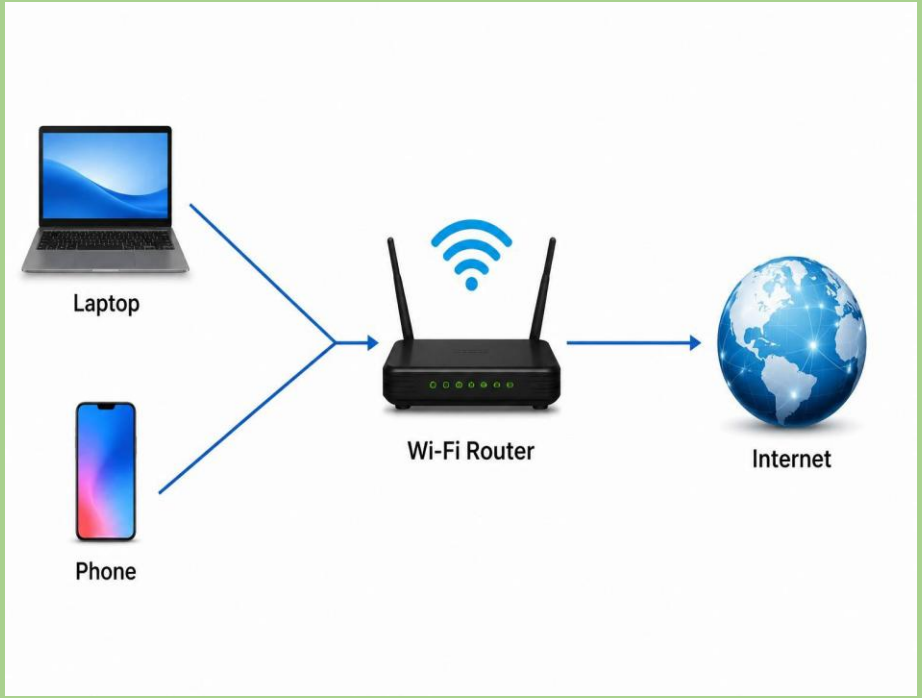
Computer networks enhance system performance, availability, and productivity.

- Backup and recovery systems
- Distributed processing
- Load sharing among devices
- Centralized management and maintenance

Example of a Small Network

When two or more devices at home are connected through a Wi-Fi router, they form a **small computer network**.

- The **Wi-Fi router** acts as the central device that connects all devices in the network.
- The **laptop** and **phone** can communicate with each other through the router.
- The router provides access to the **Internet** for all connected devices.
- Resources such as files, printers, and media can be shared among the connected devices.



Real-Life Example:

A family uses a Wi-Fi router at home to connect a laptop, smartphone, smart TV, and printer. All devices can access the Internet and share resources through the same network.

Components of a Computer Network

A computer network consists of three main components: **End Devices**, **Connecting Devices**, and **Transmission Media**.

1. End Devices (Hosts)

End devices are the devices that **send, receive, or process data** in a network.

Examples

✓ Computers
✓ Laptops
✓ Smartphones
✓ Tablets
✓ Servers
✓ Printers

Functions

- Generate and receive data
- Communicate with other devices on the network
- Access network resources and services

2. Connecting Devices

Connecting devices are responsible for **linking devices and networks together** and directing data to its destination.

Examples

Router

- Connects different networks
- Provides Internet access

Switch

- Connects multiple devices within a Local Area Network (LAN)
- Transfers data efficiently between devices

Modem

- Connects a network to the Internet Service Provider (ISP)
- Converts digital signals to analog signals and vice versa

Access Point (AP)

- Provides wireless connectivity to devices
- Extends Wi-Fi coverage

3. Transmission Media

Transmission media refers to the **communication channel through which data travels** from one device to another.

A. Wired Media

Twisted Pair Cable

- Most commonly used network cable
- Used in Ethernet networks

Coaxial Cable

- Used in cable television and broadband Internet

Fiber Optic Cable

- Uses light signals for transmission
- Offers very high speed and long-distance communication

B. Wireless Media

Radio Waves

- Used for wireless communication over varying distances

Wi-Fi

- Enables wireless networking in homes, offices, and public places

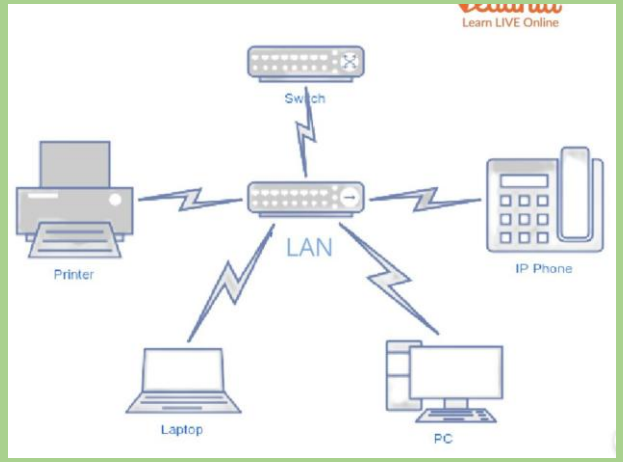
Satellite Communication

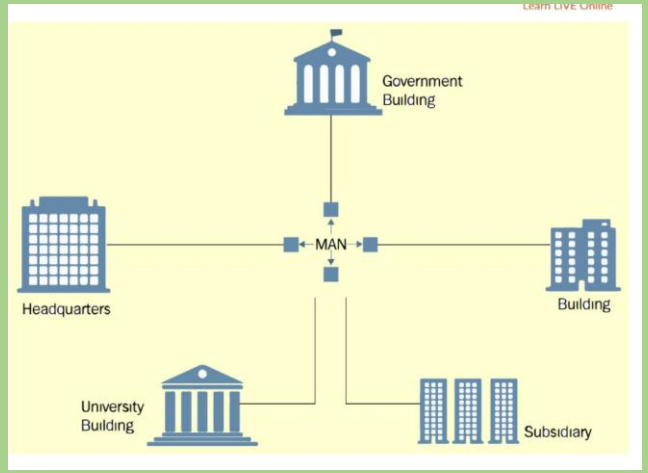
- Used for long-distance communication and remote-area connectivity

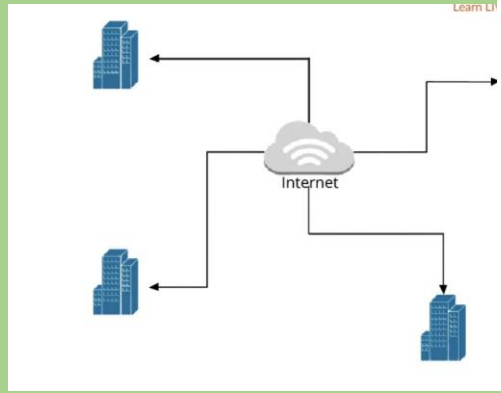
TYPES OF NETWORK

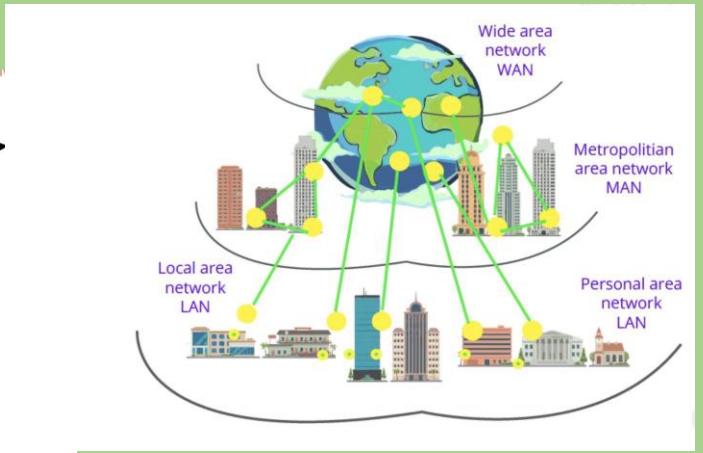
Network Type	Full Form	Area of Coverage	Example
PAN	Personal Area Network	A few meters (around 1–10 m)	Bluetooth connection between a phone and earbuds
LAN	Local Area Network	A room, building, or campus (up to a few km)	Network within a school, office, or home
CAN	Campus Area Network	Multiple buildings within a campus (1–10 km)	University campus network
MAN	Metropolitan Area Network	A city or metropolitan area (10–100 km)	City-wide cable TV or internet network
WAN	Wide Area Network	Countries or continents (100 km and above)	Bank networks connecting branches worldwide
GAN	Global Area Network	Worldwide coverage	The Internet


PAN


Learn LIVE Online
Switch
LAN
Printer
IP Phone
Laptop
PC


Government Building
MAN
Headquarters
Building
University Building
Subsidiary


Internet


Wide area network WAN
Metropolitian area network MAN
Local area network LAN
Personal area network LAN

Intranet

- An intranet is a private network that can only be accessed by authorized users.
- The prefix "intra" means "internal" and therefore implies an intranet is designed for internal communications.
- An **Intranet** is a private network used within an organization to securely share information, resources, and communication among employees.
- It uses internet technologies but is accessible only to authorized users within the organization.
- **Example:** Employee portal, internal company website, school management system.

1.1 OVERVIEW OF THE INTERNET

What is an Internetwork?

An **Internetwork** (or **internet**) is formed when two or more networks are connected together to allow communication between them.

These networks may be:

- LANs
- WANs
- Or a combination of both

An internetwork is a collection of interconnected networks that communicate with one another as a single system.

- Internetworking is needed to connect multiple networks across different offices, departments, and locations, enabling seamless communication and resource sharing between them.

Components of an Internetwork

An internetwork may contain:

1. LANs

Provide communication within a local area.

2. WANs

Connect distant networks over large geographical areas.

3. Routers

Connect different networks and route packets between them.

4. Switches

Forward packets within a LAN.

- The Internet is not just a single network.
- It is an **internetwork** — a network made by connecting many smaller networks together across the world.
- Millions of private, public, academic, business, and government networks are interconnected to form the Internet.

Switch Role of Routers and Switches

- Works inside a LAN
- Sends packets to the correct host
- Does not forward local traffic outside the LAN

Router

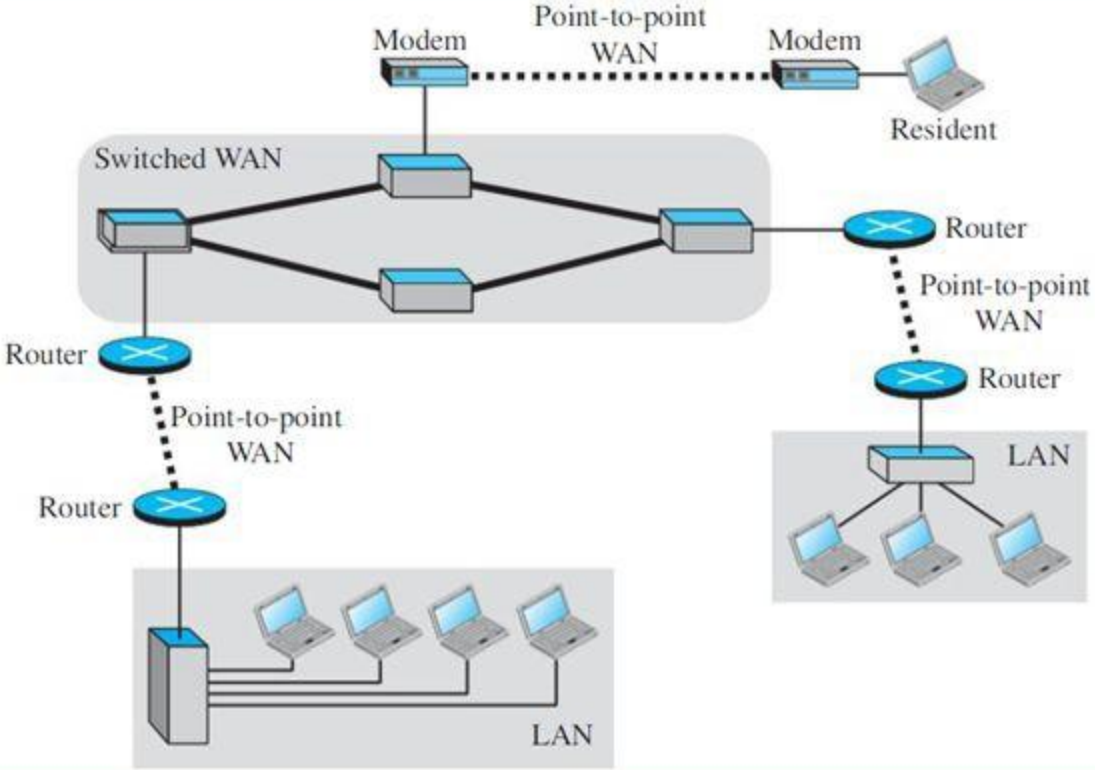
- Connects multiple networks
- Routes packets from one network to another

Communication within the same LAN:
When a computer sends data to another computer on the same local network, the **switch** delivers the data directly to the destination device, and the **router is not involved**.

Communication between different LANs:
When a computer sends data to a device on another network (such as a different office), the data travels through the **switch**, then the **router**, across the **WAN**, to another **router**, and finally through a **switch** to reach the destination device.

- **Path:**
  Host → Switch → Router → WAN → Router → Switch → Host.

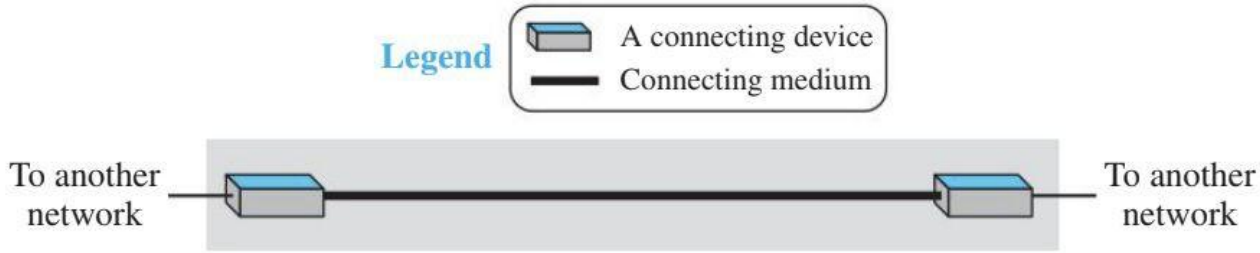
Figure 1.5 *A heterogeneous network made of four WANs and three LANs*



Point-to-Point WAN

- A **Point-to-Point WAN** directly connects **two devices or networks** using a communication link such as a cable or wireless connection.
- **Example:** A dedicated connection between two office branches.

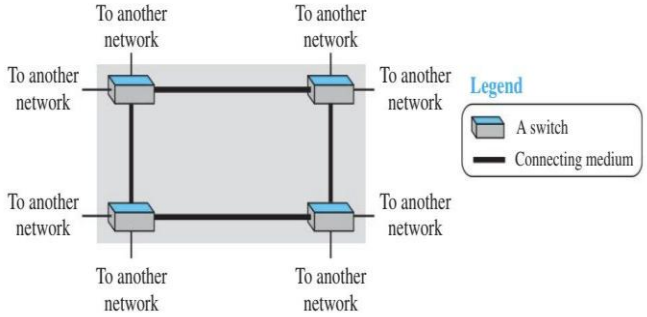
Figure 1.2 *A point-to-point WAN*



Switched WAN

- Connects **multiple devices or networks** through switches, allowing communication over a larger area.
- **Example:** The Internet and large telecommunications networks.

Figure 1.3 *A switched WAN*



Private Internet vs Internet

- **Private internet (small "i")**: A network created by connecting a company's or organization's internal networks. It is private and accessible only to authorized users.
- **Internet (capital "I")**: The global public network that connects millions of networks and devices worldwide, accessible to anyone with an internet connection.

Example:

- A company's internal network connecting all its offices = **Private internet**
- The worldwide network used for websites, email, and online services = **Internet**.

Switched Network

Introduction

A **switched network** is a network in which a device called a **switch** connects two or more communication links together.

The switch receives data from one link and forwards it to another link whenever needed.

Definition

A switched network is a communication network in which switches are used to forward data from one link to another.

Role of a Switch

A switch performs the task of:

- Receiving data
- Determining the destination
- Forwarding the data to the correct link

In modern communication systems, switching helps multiple users communicate efficiently.

Types of Switched Networks

There are two main types of switched networks:

1. **Circuit-Switched Network**
2. **Packet-Switched Network**

---

1. Circuit-Switched Network

Introduction

In a **circuit-switched network**, a dedicated communication path called a **circuit** is established between two end systems before communication begins.

The path remains reserved during the entire communication session.

Working Principle

- A fixed path is created between sender and receiver
- Resources are reserved for the whole communication period
- The connection can either be:
  - Active
  - Inactive

The switch only activates or deactivates the circuit.

Example

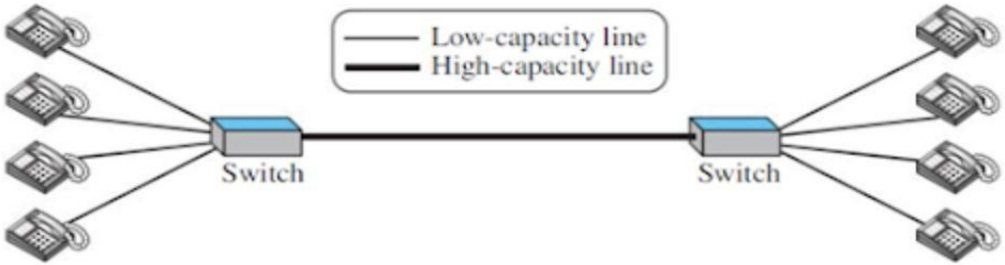
Traditional telephone networks are classic examples of circuit switching.

```
Telephone ---- Switch ===== Switch ---- Telephone
```

Once a call is established:

- The communication path remains dedicated
- Other users cannot use that reserved capacity

Figure 1.6 *A circuit-switched network*



Circuit-switched network creates a **dedicated communication path (circuit)** between two devices before communication begins. Once the circuit is established, the entire path remains reserved for those users until the communication ends.

Example from the Figure

- There are **4 telephones on each side** connected through switches.
- The two switches are connected by a **high-capacity line**.
- This high-capacity line can support **4 simultaneous voice calls**.
- The switches only **forward** the voice signals; they do not store data.

Case 1: Full Utilization

- All 4 telephones on one side are connected to 4 telephones on the other side.
- The high-capacity line carries **4 voice calls simultaneously**.
- The entire capacity of the line is used.
- This is the **most efficient** situation.

Case 2: Partial Utilization

- Only 1 telephone is in use.
- Only **1 out of 4 available channels** is used.
- The remaining **3 channels stay unused**, even though they are reserved and available.
- Only **25% of the line's capacity** is utilized.
- This makes circuit switching inefficient during low traffic.

2. Packet-Switched Network

Introduction

In a **packet-switched network**, data is divided into small units called **packets**.

Each packet is transmitted independently through the network.

Working Principle

Instead of continuous communication:

- Data is broken into packets
- Packets are stored temporarily
- Then forwarded toward the destination

This method is called:

Store-and-Forward Transmission

Packet-Switched Network

In a **packet-switched network**, data is not sent as one continuous stream. Instead, it is divided into **small units called packets**.

Each packet:

- Contains a portion of the data.
- Travels independently through the network.
- Can be **stored temporarily** and **forwarded later** if needed.

Components

- **Computers** act as end devices.
- **Routers** act as switches.
- Routers have **memory (queues)** to store packets temporarily.

How It Works

Case 1: Low Traffic

- One computer at Site A communicates with one computer at Site B.
- The communication line has enough capacity.
- Packets are forwarded immediately.
- **No waiting or delay occurs.**

Case 2: Heavy Traffic

- Many computers send packets at the same time.
- The communication line reaches its maximum capacity.
- New packets arriving at the router cannot be sent immediately.
- They are placed in a **queue (buffer)**.
- The router forwards them **one by one in the order they arrived**.

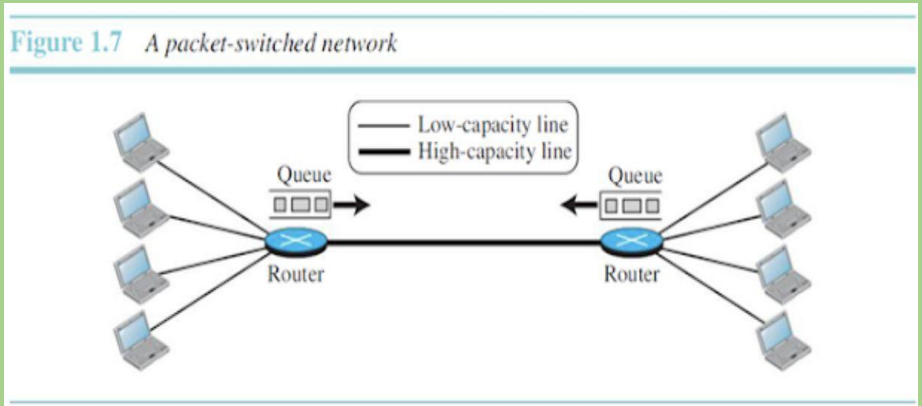
Example

Suppose:

- The main communication line can carry only **2 packets at a time**.
- Four computers start sending packets simultaneously.

Then:

1. The first packets are transmitted immediately.
2. The remaining packets wait in the router's queue.
3. As space becomes available, queued packets are forwarded.



Why Is Packet Switching More Efficient?

- No dedicated path is reserved.
- Bandwidth is shared among all users.
- Resources are used only when data is actually being transmitted.
- Unused capacity can be utilized by other users.

Introduction to the Internet

- An **internet** (small "i") is a collection of two or more interconnected networks that can communicate with each other.
- The **Internet** (capital "I") is the world's largest and most well-known internetwork.
- The **Internet** is a global network of interconnected computer networks that enables communication, information exchange, and resource sharing among millions of users worldwide.

Structure of the Internet

The Internet is organized into three levels:

1. **Backbone Networks**
2. **Provider Networks**
3. **Customer Networks**

Structure of the Internet

The Internet has three levels:

1. Backbone Networks

- These are the **largest and fastest networks**.
- Owned by major telecommunication companies such as:AT&T ,Verizon,Sprint,NTT
- They form the **core of the Internet**.
- Backbone networks are connected through **peering points**.

2. Provider Networks

- Smaller networks that connect to backbone networks.
- They pay fees to use the backbone services.
- They provide Internet access to customers.
- Also called **National or Regional ISPs**.

3. Customer Networks

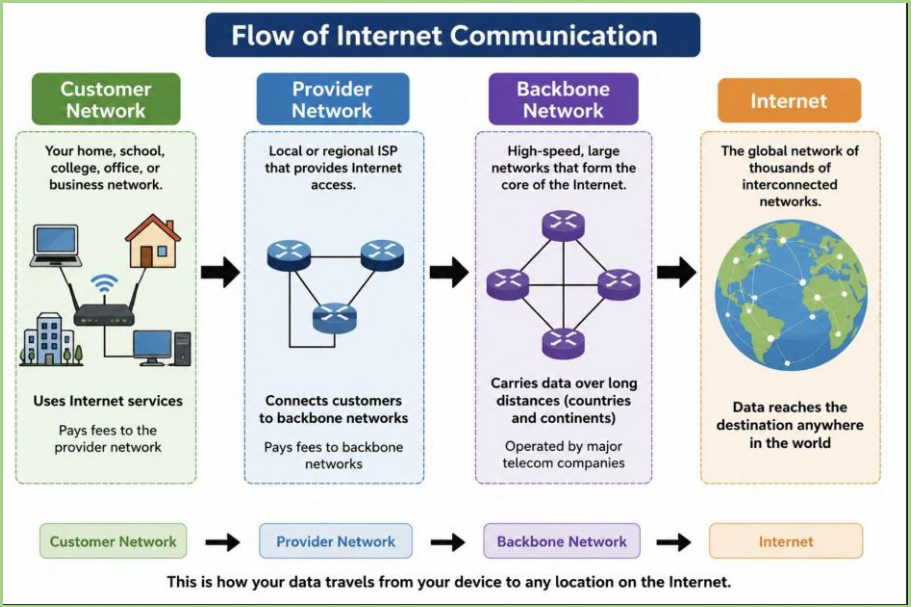
- Networks used by homes, schools, colleges, offices, and businesses.
- They are located at the **edge of the Internet**.
- They pay provider networks for Internet services.

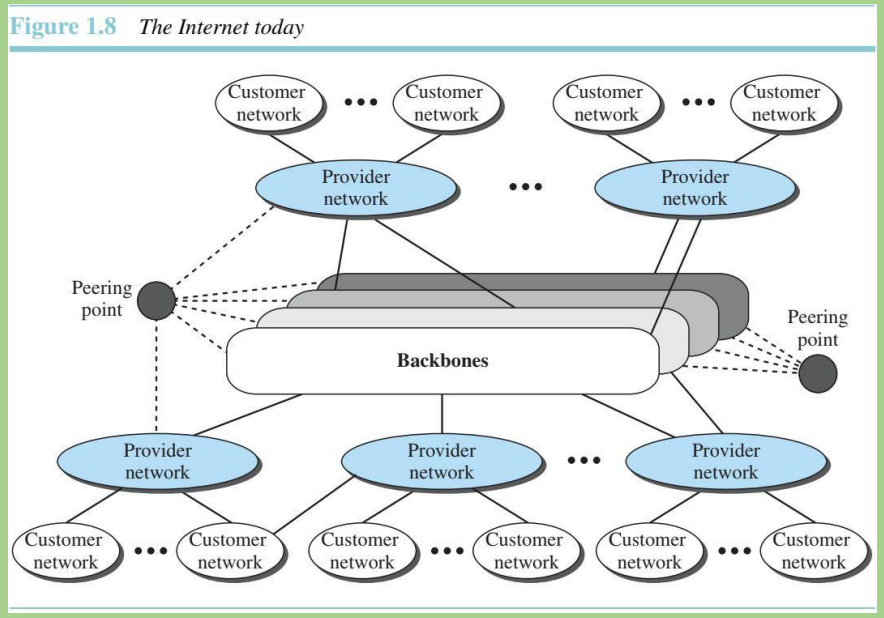
ISP (Internet Service Provider)

Both backbone networks and provider networks are called **Internet Service Providers (ISPs)**.

Types of ISPs

- **International ISPs** → Backbone Networks
- **National/Regional ISPs** → Provider Networks





Accessing the Internet

To access the Internet, a user must be connected to an **Internet Service Provider (ISP)**.

1. Using Telephone Networks

a) Dial-up Service

- Uses a **modem** to convert data into voice signals.
- Connects to the ISP through a telephone line.
- Very slow.
- Telephone and Internet cannot be used at the same time.

b) DSL (Digital Subscriber Line)

- Provides higher Internet speed.
- Uses existing telephone lines.
- Telephone and Internet can be used simultaneously.

2. Using Cable Networks

- Uses cable TV networks for Internet access.
- Provides higher speed than dial-up and DSL.
- Speed may vary depending on the number of users sharing the cable.

3. Using Wireless Networks

- Uses wireless technologies to connect to the Internet.
- Suitable for homes and small businesses.
- Increasingly popular due to mobility and convenience.

4. Direct Connection to the Internet

- Used by large organizations, universities, and corporations.
- They lease a high-speed WAN connection and connect directly to a regional ISP.
- Can act as a local ISP for their internal networks.

Protocol Layering

Introduction

In computer networks, communication between devices follows a set of rules called a **protocol**.

A protocol defines:

- How data is transmitted
- How devices communicate
- How errors are handled
- How information is interpreted

When communication becomes complex, a single protocol is not enough. The communication task is therefore divided into multiple smaller tasks organized into different layers. This concept is called:

Protocol Layering

Definition

> *Protocol layering is a method of dividing a communication task into several smaller and simpler layers, where each layer performs a specific function and follows its own protocol.*

Why Protocol Layering is Needed

Modern communication systems are complex because they involve:

- Data transmission
- Error handling
- Encryption
- Routing
- Delivery mechanisms

Instead of handling all tasks together, networking divides the work into layers.

Each layer:

- Performs a specific task
- Provides services to the upper layer
- Receives services from the lower layer

First Scenario: Single-Layer Communication

In this :

- Maria and Ann communicate face-to-face
- Communication happens directly in one layer

Figure 1.9 *A single-layer protocol*

Even in simple communication, certain rules are followed:

- Greeting each other
- Speaking one at a time
- Using understandable language
- Taking turns in conversation

These rules form a simple protocol.

Second Scenario: Three-Layer Communication

When Ann moves to another city, communication becomes more complex.

Maria and Ann now communicate through postal mail and use encryption for security.

Their communication is divided into three layers.

Maria first creates a message in plain English at the third layer. The message is then passed to the second layer, where it is encrypted into ciphertext. Next, the first layer places the encrypted message in an envelope, adds the addresses, and sends it through the postal system.

At Ann's side, the first layer receives the mail and extracts the ciphertext from the envelope. The second layer decrypts the ciphertext and converts it back into plaintext. Finally, the third layer reads the message as if Maria were speaking directly to Ann.

This example shows how protocol layering divides communication into smaller tasks such as:

- Message creation
- Encryption/decryption
- Sending and receiving mail

Each layer performs a specific function independently.

Three Layers in the Example

Layer	Function
Layer 3	Listen and Talk
Layer 2	Encrypt and Decrypt
Layer 1	Send and Receive Mail

Working of the Three-Layer Protocol

At Maria's Side

Layer 3

- Maria creates the message (plaintext)

Layer 2

- Message is encrypted into ciphertext

Layer 1

- Ciphertext is placed in an envelope and mailed

At Ann's Side

Layer 1

- Mail is received

Layer 2

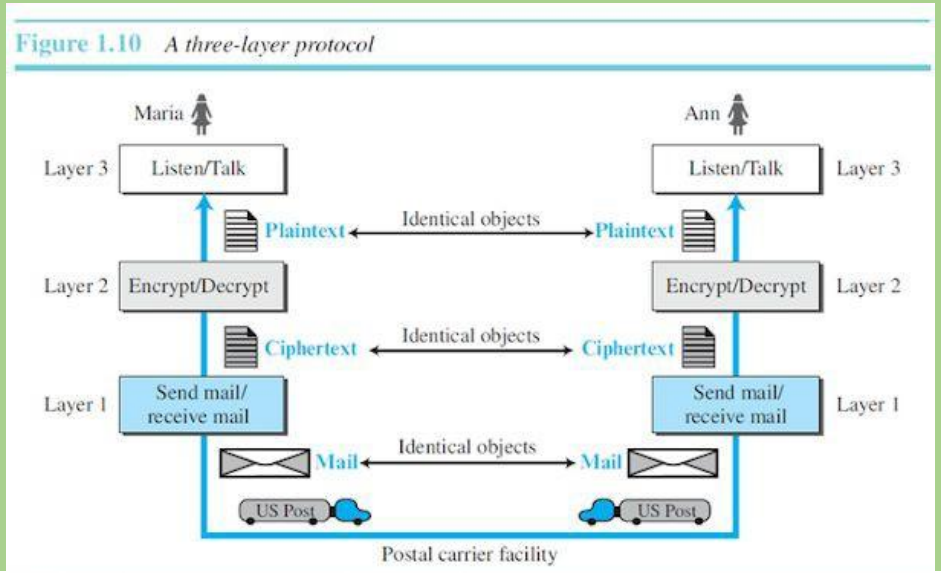
- Ciphertext is decrypted into plaintext

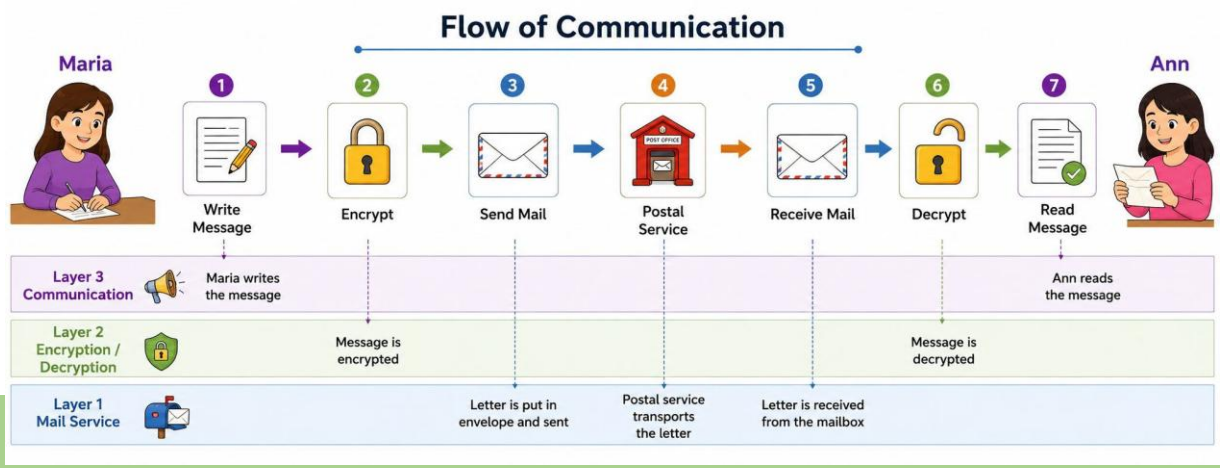
Layer 3

- Ann reads the message

Simple Representation

```
Maria
Layer 3 → Plaintext
Layer 2 → Encryption
Layer 1 → Mail Delivery
        ↓
Postal System
        ↓
Ann
Layer 1 → Receive Mail
Layer 2 → Decryption
Layer 3 → Read Message
```





Principles of Protocol Layering

1. Bidirectional Communication

Each layer must support communication in both directions.

Examples:

- Listen / Talk
- Encrypt / Decrypt
- Send / Receive

---

2. Identical Objects at Peer Layers

Objects exchanged between corresponding layers should be identical.

Examples:

- Layer 3 → Plaintext
- Layer 2 → Ciphertext
- Layer 1 → Mail

Logical Connections

In protocol layering:

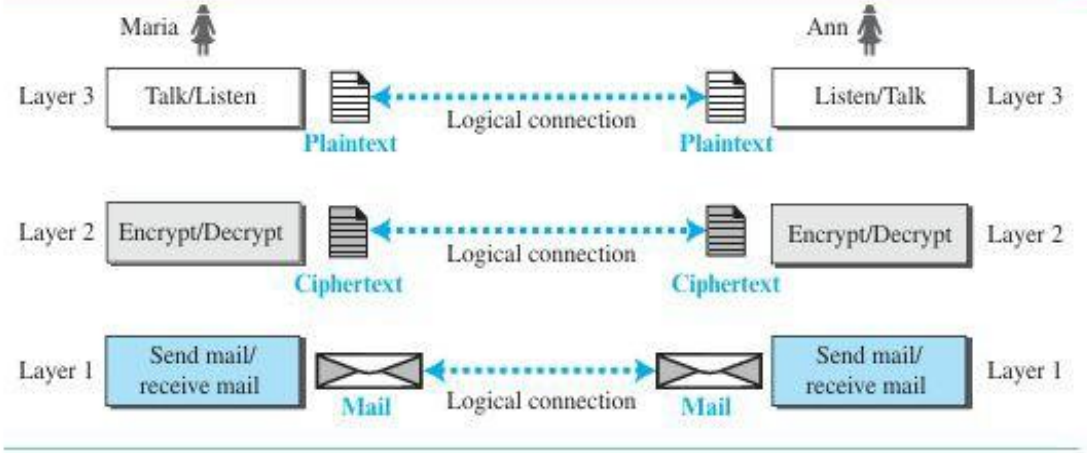
- Each layer appears to communicate directly with its corresponding layer at the other side.

This is called:

Logical Layer-to-Layer Communication

Although actual data travels through lower layers, logically each layer communicates with its peer layer.

Figure 1.11 *Logical connection between peer layers*



TCP/IP Protocol Suite

Introduction

The **TCP/IP Protocol Suite** is the fundamental set of communication protocols used for data transmission over the Internet and most modern computer networks.

TCP/IP stands for:

- **Transmission Control Protocol (TCP)** – Ensures reliable and error-free delivery of data between devices.
- **Internet Protocol (IP)** – Provides addressing and routing mechanisms to deliver data packets across networks.

TCP/IP consists of a collection of protocols organized into multiple layers, where each layer performs specific communication functions and works together to enable seamless data exchange.

Definition

The **TCP/IP Protocol Suite** is a hierarchical collection of communication protocols organized into layers that facilitate reliable and efficient communication between interconnected devices over the Internet.

Why TCP/IP Uses Layering

Network communication is a complex process involving data transmission, routing, error checking, and application services. To simplify these tasks, TCP/IP adopts a **layered architecture**.

Advantages of Layering

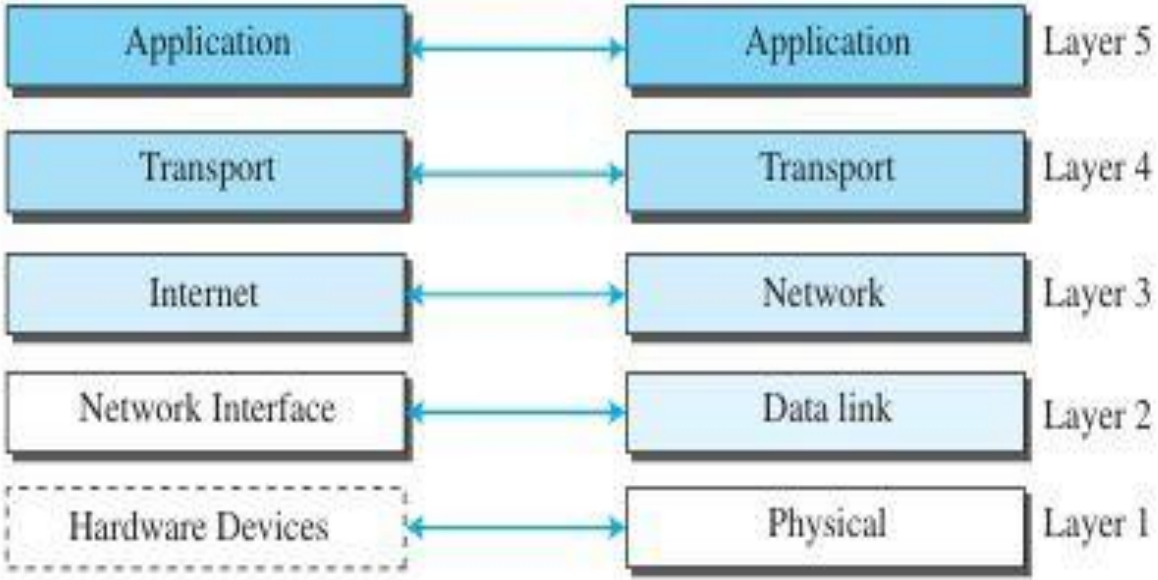
- Each layer performs a specific and well-defined function.
- Layers operate independently of one another.
- Network design, implementation, and troubleshooting become easier.
- Modifications in one layer have minimal impact on other layers.
- Standardization enables interoperability among different devices and technologies.

Layer Interaction

Each layer provides services to the layer above it and receives services from the layer below it. This structured approach ensures efficient, reliable, and organized communication across networks.

In simple terms, the TCP/IP model divides a complex communication process into smaller, manageable layers, making Internet communication efficient and reliable.

Figure 1.12 *Layers in the TCP/IP protocol suite*



TCP/IP Layered Architecture

The **TCP/IP Protocol Suite** follows a layered architecture to simplify network communication. Originally, TCP/IP was designed as a **four-layer model**, but it is commonly represented today as a **five-layer model** by separating the Data Link and Physical layers.

Five Layers of TCP/IP

Layer	Function
Application Layer	Provides network services directly to user applications and enables interaction between software and the network.
Transport Layer	Ensures end-to-end communication, data reliability, flow control, and error recovery between hosts.
Network Layer	Handles logical addressing, routing, and delivery of packets from source to destination across interconnected networks.
Data Link Layer	Provides node-to-node communication, framing, error detection, and access to the physical medium.
Physical Layer	Transmits raw bits over the communication medium using electrical, optical, or wireless signals.

Layer	Function	Responsibilities	Protocols / Devices	Examples
Application Layer	Provides network services directly to user applications.	User communication, data formatting, network services, resource sharing.	HTTP, HTTPS, FTP, SMTP, DNS	Web browsing, Email, File transfer, Domain name lookup
Transport Layer	Provides end-to-end process-to-process communication between hosts.	Segmentation, error control, flow control, reliable delivery, multiplexing.	TCP, UDP	File transfer (TCP), Web browsing (TCP), Video streaming (UDP), Online gaming (UDP)
Network Layer	Handles routing and delivery of packets across networks.	Logical addressing, routing, packet forwarding, path determination.	IP (IPv4/IPv6), Routers	Sending data between different networks, Internet communication
Data Link Layer	Provides communication between devices on the same network link.	Framing, physical addressing (MAC), error detection, media access control.	Ethernet, Wi-Fi, Switches	Communication within a LAN, Frame transmission between devices
Physical Layer	Transmits raw bits through the transmission medium.	Signal transmission, bit representation, cable and connector specifications.	Cables, Hubs, Repeaters, Fiber Optics	Electrical signals through cables, Wireless radio signals, Optical transmission through fiber

Communication in TCP/IP

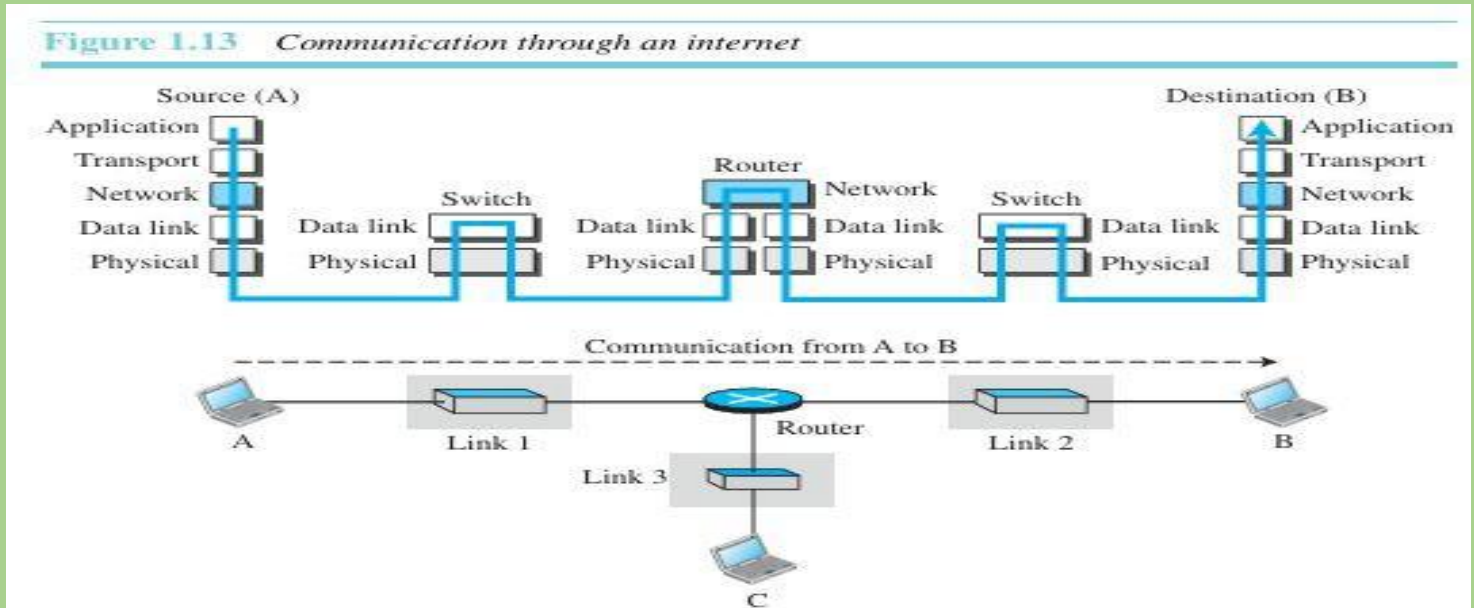
Communication Process

Consider a scenario where **Computer A** sends data to **Computer B** through a switch, a router, and another switch.

- At the sender (Computer A), data passes through all five TCP/IP layers from top to bottom.
- The router processes the data at the Physical, Data Link, and Network layers.
- The switches process the data at the Physical and Data Link layers.
- At the receiver (Computer B), data passes through all five layers from bottom to top.

Role of Devices in TCP/IP

Device	Layers Used	Function
Host Computer	Application, Transport, Network, Data Link, Physical	Generates and receives data
Router	Network, Data Link, Physical	Routes packets between networks
Switch	Data Link, Physical	Forwards frames within a LAN



Role of Devices in TCP/IP

Device	Layers Used	Function
Host Computer	Application, Transport, Network, Data Link, Physical	Generates and receives data
Router	Network, Data Link, Physical	Routes packets between networks
Switch	Data Link, Physical	Forwards frames within a LAN

End-to-End and Hop-to-Hop Communication

Communication Type	Layers Involved	Description
End-to-End Communication	Application, Transport, Network	Communication between source and destination hosts across the Internet.
Hop-to-Hop Communication	Data Link, Physical	Communication between two directly connected devices.

End-to-End Communication

The upper three layers provide communication from the source host to the destination host:

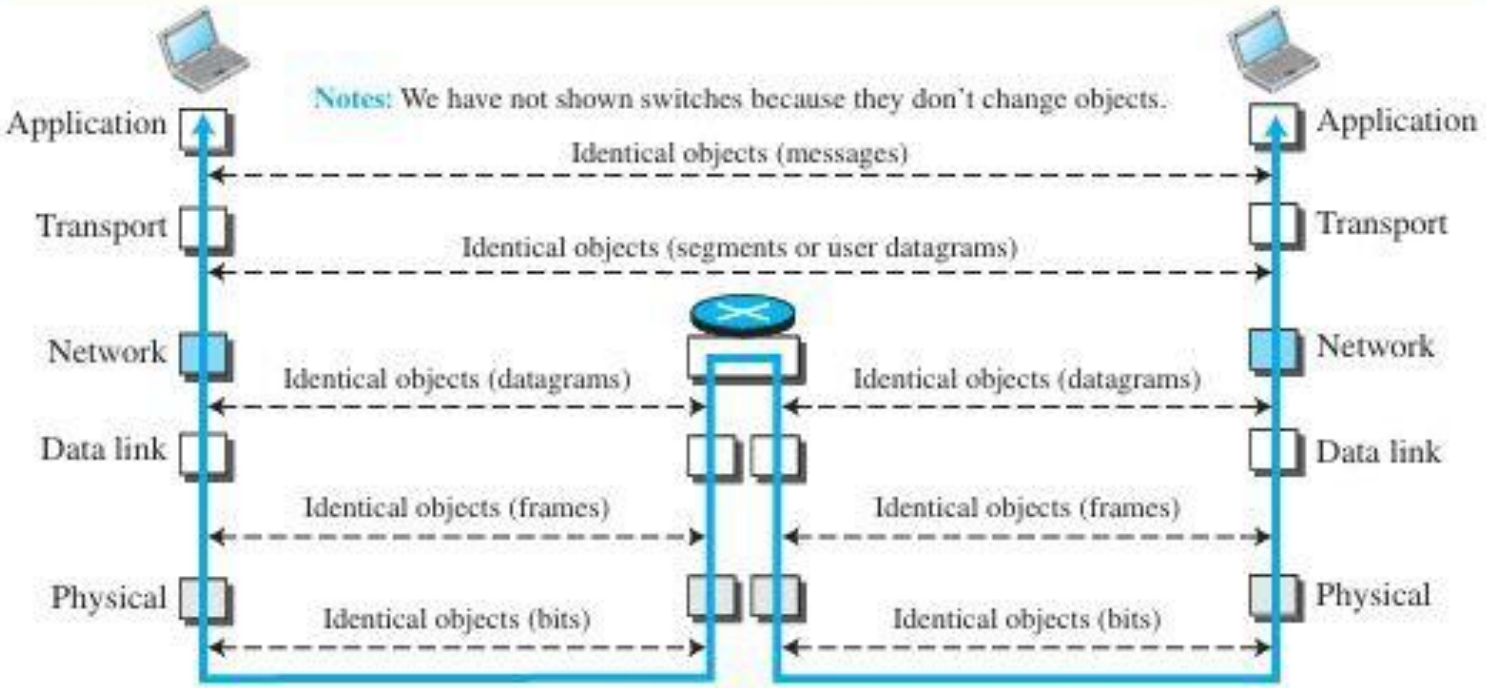
- Application Layer
- Transport Layer
- Network Layer

Hop-to-Hop Communication

The lower two layers provide communication between neighboring devices:

- Data Link Layer
- Physical Layer

Figure 1.15 *Identical objects in the TCP/IP protocol suite*



Identical Objects in the TCP/IP Protocol Suite

This figure illustrates the concept of **logical communication** in the TCP/IP model. Although data physically travels through different layers and devices, each layer on the sender appears to communicate directly with its corresponding layer on the receiver.

1. Application Layer

- The sender creates a **message**.
- Logically, the Application Layer communicates with the Application Layer of the destination host.
- The object exchanged is called a **Message**.

2. Transport Layer

- The message is divided into **segments** (TCP) or **user datagrams** (UDP).
- Logical communication occurs between the Transport Layers of the source and destination hosts.
- The object exchanged is a **Segment/User Datagram**.

3. Network Layer

- The Transport Layer passes data to the Network Layer.
- The Network Layer adds an IP header, creating a **Datagram (Packet)**.
- Routers process data only up to this layer.
- The datagram remains logically the same throughout the Internet.

4. Data Link Layer

- The Network Layer passes the datagram to the Data Link Layer.
- The Data Link Layer encapsulates it into a **Frame**.
- Frames are exchanged only between directly connected devices.
- At every hop, a new frame may be created.

5. Physical Layer

- Frames are converted into **Bits**.
- Bits are transmitted as electrical, optical, or wireless signals
- through the medium.

Data Units in TCP/IP

Each layer adds its own header and creates a specific data unit.

Layer	Data Unit
Application Layer	Message
Transport Layer	Segment (TCP) / User Datagram (UDP)
Network Layer	Datagram (Packet)
Data Link Layer	Frame
Physical Layer	Bits

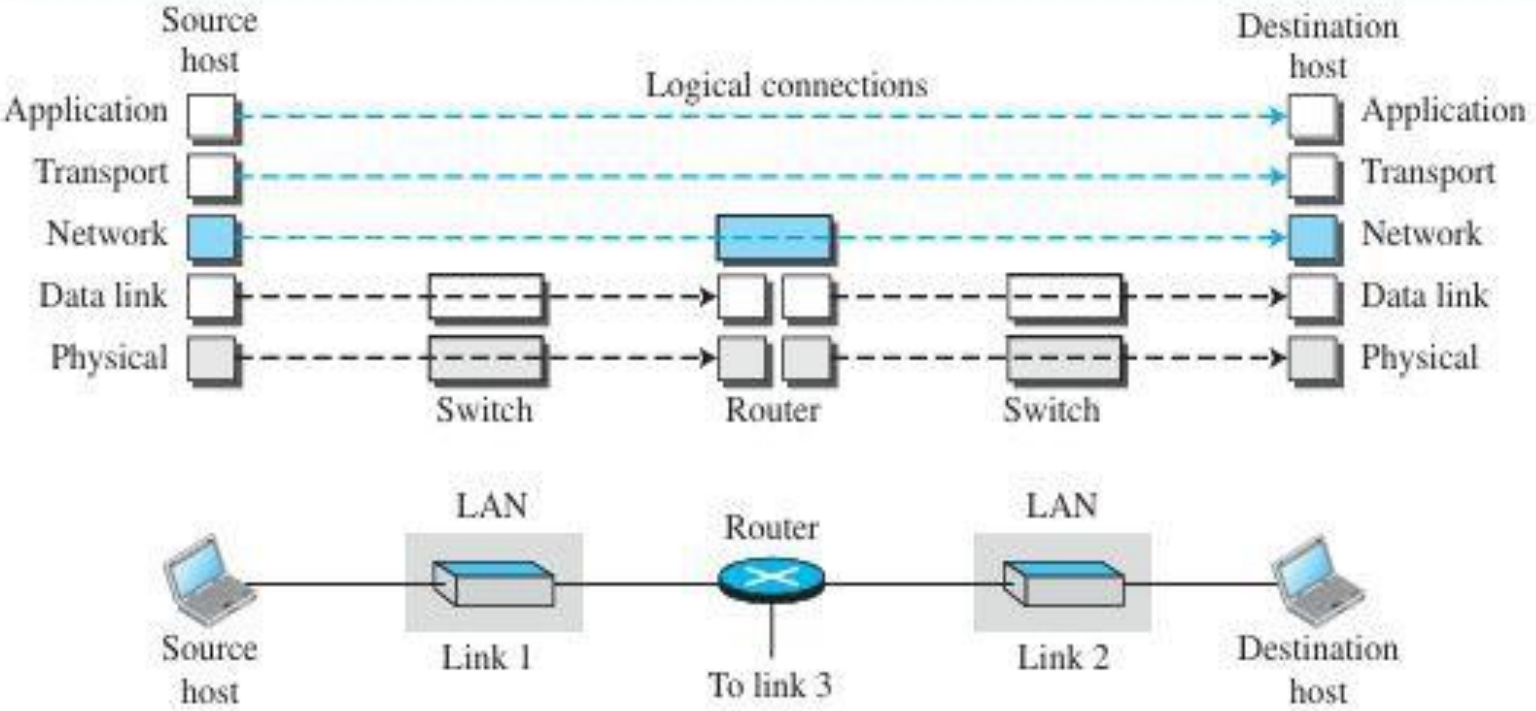
Logical Layer-to-Layer Communication

In a layered architecture, each layer on the source host communicates **logically** with the corresponding layer on the destination host.

Source Layer	Destination Layer
Application Layer	Application Layer
Transport Layer	Transport Layer
Network Layer	Network Layer
Data Link Layer	Data Link Layer
Physical Layer	Physical Layer

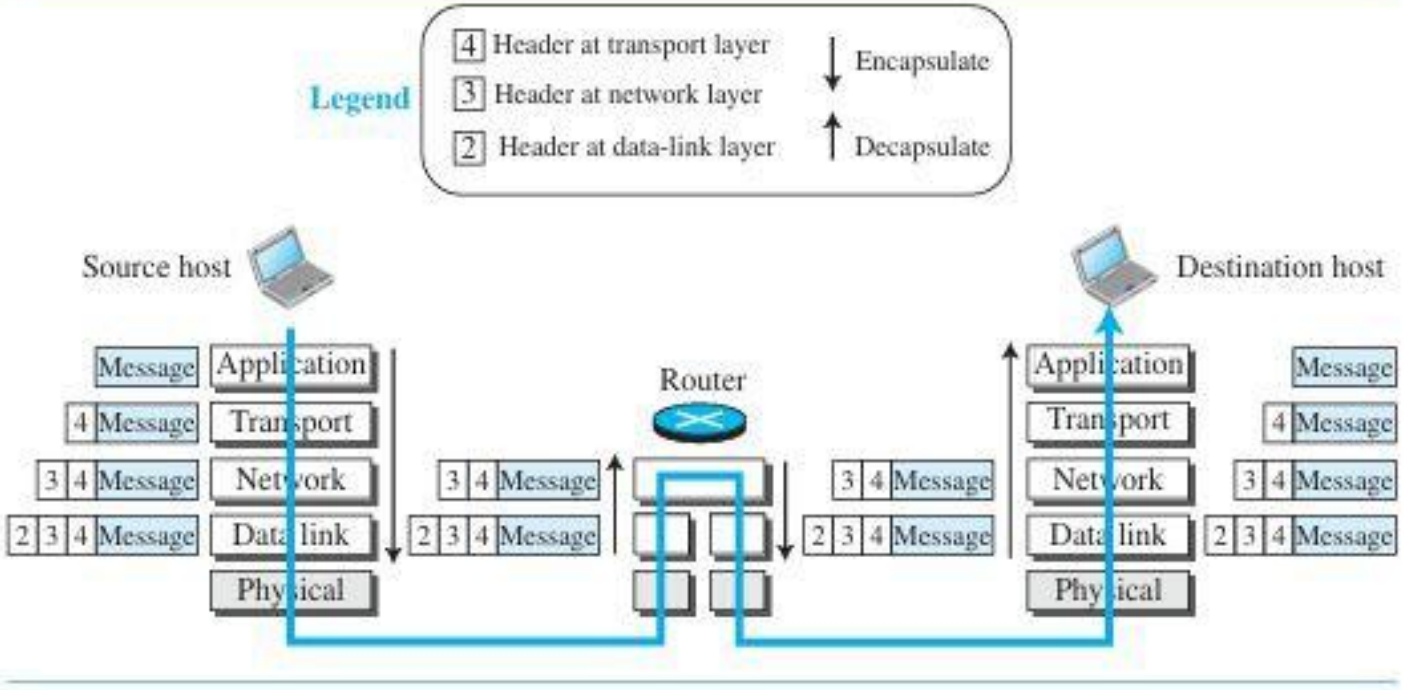
This concept is known as **Logical Layer-to-Layer Communication**.

Figure 1.14 *Logical connections between layers of the TCP/IP protocol suite*



Layer	Main Duty	Examples of Protocols
Application Layer	Provides communication between application programs (process-to-process communication). It allows users to access network services such as web browsing, email, and file transfer.	HTTP, FTP, SMTP, DNS, Telnet, SSH, SNMP
Transport Layer	Provides end-to-end communication between applications. It performs segmentation, reliability, flow control, error control, and congestion control.	TCP, UDP, SCTP
Network Layer	Provides host-to-host communication and routes packets across multiple networks. It handles logical addressing and routing.	IP, ICMP, IGMP, DHCP
Data Link Layer	Transfers packets across a single network link. It performs framing, physical addressing, and error detection.	Ethernet, Wi-Fi, PPP
Physical Layer	Transmits raw bits as electrical, optical, or wireless signals through the transmission medium.	Fiber Optics, Twisted Pair Cable, Wireless Technologies

Encapsulation and Decapsulation



Encapsulation and Decapsulation in TCP/IP

Encapsulation at the Source Host

1. The **Application Layer** creates the data called a **Message**.
2. The **Transport Layer** receives the message and adds a **Transport Header**, creating a **Segment (TCP)** or **User Datagram (UDP)**.
3. The **Network Layer** adds an **IP Header** to the segment, creating a **Datagram (Packet)**.
4. The **Data Link Layer** adds a **Frame Header**, creating a **Frame**.
5. The **Physical Layer** converts the frame into **Bits** and transmits them through the communication medium.

Encapsulation Flow

Message → Segment/User Datagram → Datagram → Frame → Bits

Decapsulation and Encapsulation at a Router

1. The router receives the frame and removes the **Data Link Header** (Decapsulation).
2. The router examines the **Destination IP Address** in the datagram.
3. The router determines the next hop using its routing table.
4. The datagram is then passed to the outgoing Data Link Layer.
5. A new frame is created by adding a new Data Link Header (Encapsulation).
6. The frame is transmitted to the next device.

Decapsulation at the Destination Host

1. The **Physical Layer** receives the bits.
2. The **Data Link Layer** removes the frame header and extracts the datagram.
3. The **Network Layer** removes the IP header and extracts the segment.
4. The **Transport Layer** removes the transport header and extracts the message.
5. The **Application Layer** receives the original message.

Decapsulation Flow

Bits → Frame → Datagram → Segment/User Datagram → Message

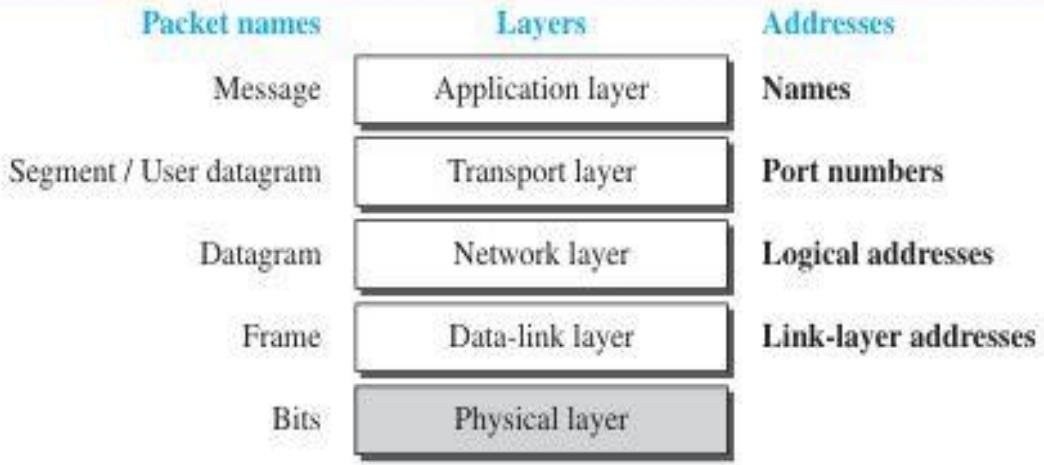
Addressing in TCP/IP

Communication between two devices requires a **source address** and a **destination address**. In the TCP/IP model, different layers use different types of addresses.

Types of Addresses in TCP/IP

Layer	Address Type	Purpose	Packet Name
Application Layer	Names	Identify users or services	Message
Transport Layer	Port Numbers	Identify application processes	Segment / User Datagram
Network Layer	Logical Addresses (IP Address)	Identify hosts on the Internet	Datagram
Data Link Layer	Link-Layer (MAC) Addresses	Identify devices on a local network	Frame
Physical Layer	No Address	Transmits bits	Bits

Figure 1.17 *Addressing in the TCP/IP protocol suite*



Examples

- **Application Layer:** www.google.com ↗, user@gmail.com ↗
- **Transport Layer:** Port 80 (HTTP), Port 25 (SMTP)
- **Network Layer:** 192.168.1.10, 172.16.0.5
- **Data Link Layer:** 00:1A:2B:3C:4D:5E (MAC Address)

Multiplexing and Demultiplexing

Multiplexing

1. Multiplexing occurs at the **source (sender)**.
2. A protocol at a layer can receive data from multiple higher-layer protocols.
3. The protocol encapsulates the data and forwards it to the lower layer.
4. A header field is used to identify the protocol from which the data originated.

Examples

- **Transport Layer**
  - TCP can receive data from HTTP, FTP, SMTP, DNS, etc.
  - UDP can receive data from DNS, DHCP, VoIP, etc.
- **Network Layer**
  - IP can receive:
    - TCP Segments
    - UDP User Datagrams
    - ICMP Packets
    - IGMP Packets
- **Data Link Layer**
  - A frame can carry:
    - IP Datagrams
    - ARP Packets
    - Other network-layer protocols

Demultiplexing

1. Demultiplexing occurs at the **destination (receiver)**.
2. A protocol examines the header information of the received packet.
3. Based on the protocol identifier, the packet is delivered to the correct higher-layer protocol.
4. The packet is then decapsulated and passed upward.

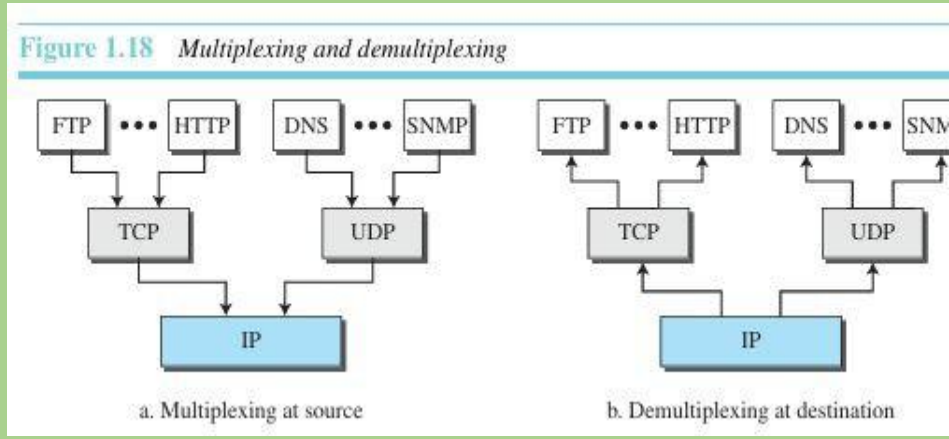


Figure 1.18 *Multiplexing and demultiplexing*

Examples

- **Data Link Layer**
  - Delivers payload to IP or ARP.
- **Network Layer**
  - Delivers packets to TCP, UDP, ICMP, or IGMP.
- **Transport Layer**
  - Delivers data to the correct application such as HTTP, FTP, SMTP, or DNS using port numbers.

OSI Model

Introduction

1. The **OSI (Open Systems Interconnection) Model** was developed by the **International Organization for Standardization (ISO)** in the late 1970s.
2. ISO is an international organization that develops worldwide standards.
3. OSI is a **reference model**, not a protocol.
4. It provides a framework for designing and understanding network communication.
5. The main goal of the OSI model is to enable communication between different computer systems regardless of their hardware and software architecture.
6. The OSI model uses a **layered architecture** with seven layers.

Definition

- The **OSI Model** is a seven-layer reference model that standardizes network communication and enables interoperability between different computer systems.

Figure 1.19 *The OSI model*

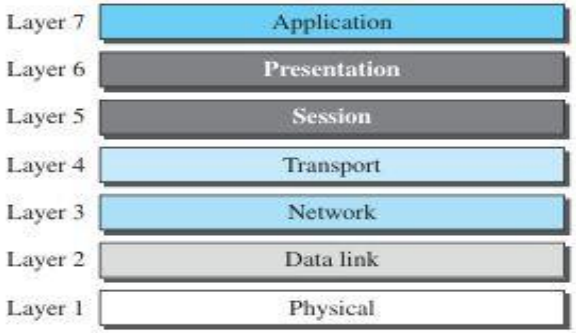
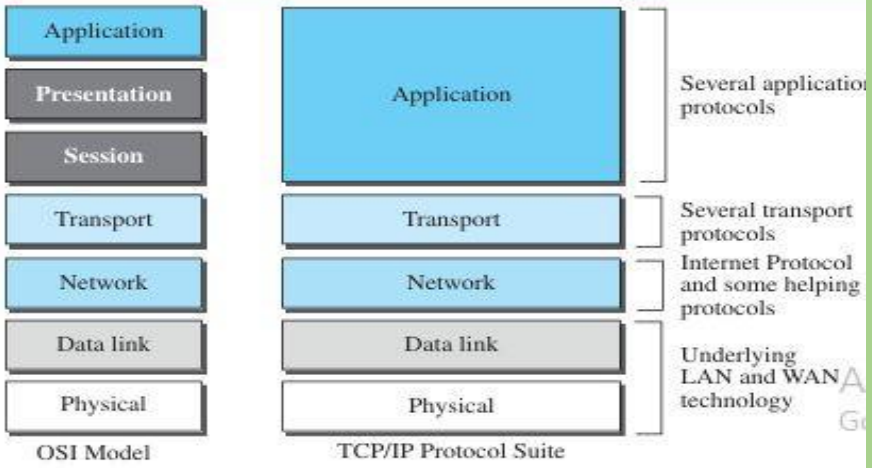


Figure 1.20 *TCP/IP and OSI model*



1. **Application Layer (Layer 7)**
   - Provides services directly to users and applications.
   - Examples: HTTP, FTP, SMTP, DNS
2. **Presentation Layer (Layer 6)**
   - Handles data translation, encryption, and compression.
   - Ensures data is presented in a usable format.
3. **Session Layer (Layer 5)**
   - Establishes, manages, and terminates communication sessions between applications.
4. **Transport Layer (Layer 4)**
   - Provides end-to-end communication and reliability.
   - Examples: TCP, UDP
5. **Network Layer (Layer 3)**
   - Handles logical addressing and routing.
   - Example: IP
6. **Data Link Layer (Layer 2)**
   - Provides node-to-node communication.
   - Performs framing and error detection.
7. **Physical Layer (Layer 1)**
   - Transmits raw bits through the communication medium.

1. **Application Layer (Layer 7)**
   - Provides services directly to users and applications.
   - Examples: HTTP, FTP, SMTP, DNS
2. **Presentation Layer (Layer 6)**
   - Handles data translation, encryption, and compression.
   - Ensures data is presented in a usable format.
3. **Session Layer (Layer 5)**
   - Establishes, manages, and terminates communication sessions between applications.
4. **Transport Layer (Layer 4)**
   - Provides end-to-end communication and reliability.
   - Examples: TCP, UDP
5. **Network Layer (Layer 3)**
   - Handles logical addressing and routing.
   - Example: IP
6. **Data Link Layer (Layer 2)**
   - Provides node-to-node communication.
   - Performs framing and error detection.
7. **Physical Layer (Layer 1)**
   - Transmits raw bits through the communication medium.

OSI vs TCP/IP

Similarities

1. Both are layered models.
2. Both support network communication.
3. Both divide communication into separate functions.

Differences

OSI Model	TCP/IP Model
7 Layers	5 Layers
Reference model	Protocol suite
Developed by ISO	Developed by DARPA/DoD
Presentation and Session layers are separate	Presentation and Session functions are included in the Application layer
Mainly used for learning and design	Used in the Internet and real-world networks

OSI Model	TCP/IP Model
Application	Application
Presentation	Application
Session	Application
Transport	Transport
Network	Network
Data Link	Data Link
Physical	Physical

Why TCP/IP Does Not Have Session and Presentation Layers

1. Some Session Layer functions are handled by Transport Layer protocols.
2. Application programs can implement Presentation and Session functions if needed.
3. Combining these layers simplifies the TCP/IP architecture.

Application Layer

Definition

The **Application Layer** is the topmost layer of the TCP/IP protocol suite. It provides services directly to users and application programs and enables communication between applications running on different hosts.

1. The Application Layer provides services to the user.
2. Communication is achieved through a **logical connection** between application layers on different hosts.
3. The two application layers behave as if there is a direct connection between them.
4. In reality, data passes through all lower layers before reaching the destination.
5. Communication at this layer is **process-to-process communication**.
6. A process sends a **request** to another process and receives a **response**.

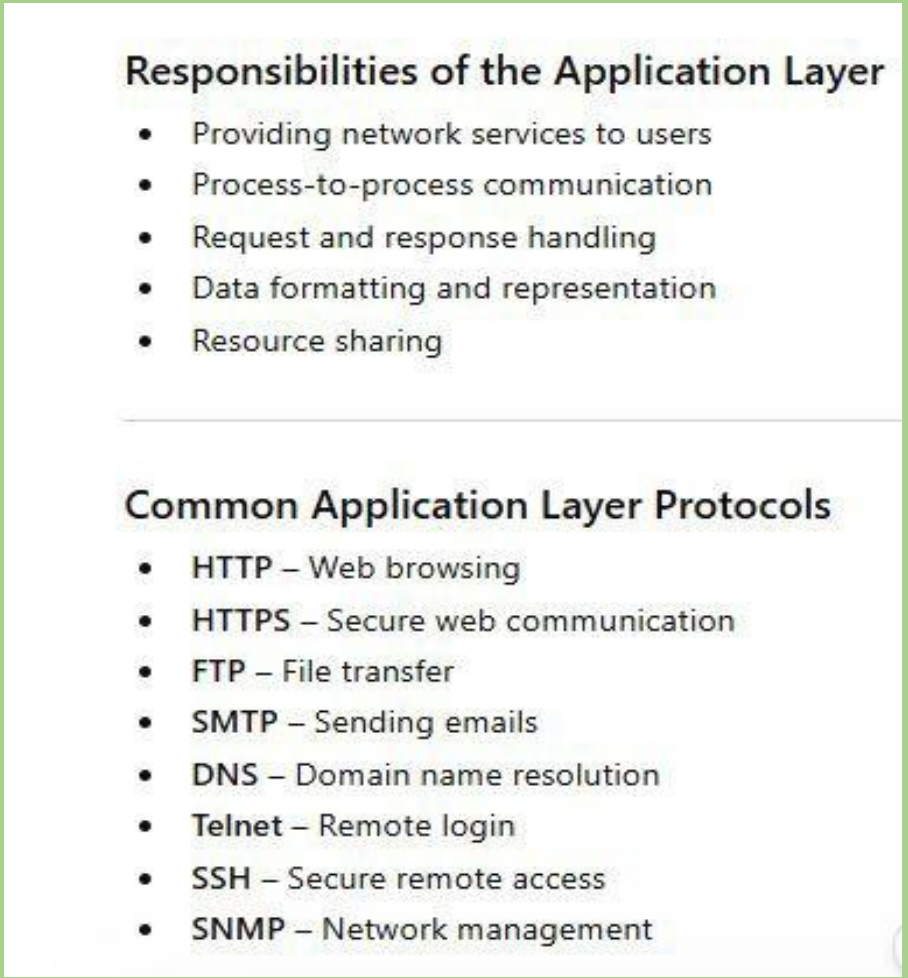
Logical Communication

- The Application Layer at the sender communicates logically with the Application Layer at the receiver.
- This communication is called **logical communication** because the connection is imaginary.
- Actual data transfer occurs through the Transport, Network, Data Link, and Physical layers.

Example

When a user opens a website:

1. The web browser sends an HTTP request.
2. The request travels through all TCP/IP layers.
3. The destination web server receives the request.
4. The server sends an HTTP response back to the browser.
5. To the applications, it appears as if they are communicating directly.



Application-Layer Paradigms

Introduction

1. Internet communication requires two application programs running on different computers.
2. These programs exchange messages through the Internet.
3. The relationship between the communicating programs is defined by an **Application-Layer Paradigm**.
4. An application program may:
    - Request a service
    - Provide a service
    - Or perform both functions
5. Two major paradigms have evolved for application-layer communication:
    - **Client-Server Paradigm**
    - **Peer-to-Peer (P2P) Paradigm**

Traditional Paradigm: Client–Server

The **Client–Server Paradigm** is a communication model in which a **server process** provides services and a **client process** requests those services through the Internet.

1. The **server** is the service provider.
2. The **client** requests services from the server.
3. The server runs continuously and waits for client requests.
4. The client starts only when a service is needed.
5. Many clients can connect to a server.
6. The roles of client and server are different and cannot be interchanged.
7. Separate client and server programs are required for each service.

Working of Client–Server Model

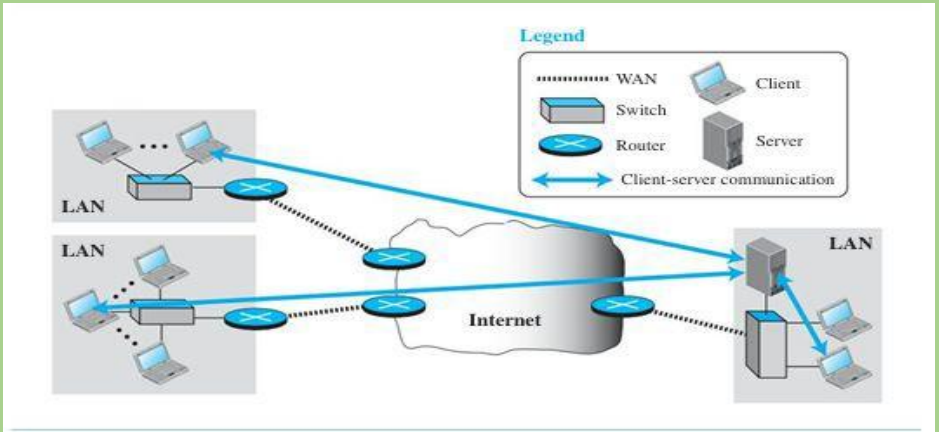
1. The client initiates a connection to the server.
2. The client sends a service request.
3. The server processes the request.
4. The server sends a response to the client.
5. The client receives the requested service.

Real-Life Example

Telephone Directory Service

- **Directory Center** → Server
- **Subscriber** → Client

The directory center is always available, while the subscriber contacts it only when information is needed.



Advantages

1. Centralized control and management.
2. Easy maintenance and updates.
3. Efficient resource sharing.
4. Better security and data management.

Disadvantages

1. Heavy workload on the server.
2. Requires powerful server hardware.
3. Server failure affects all connected clients.
4. High setup and maintenance costs.
5. Performance may degrade when many clients connect simultaneously.

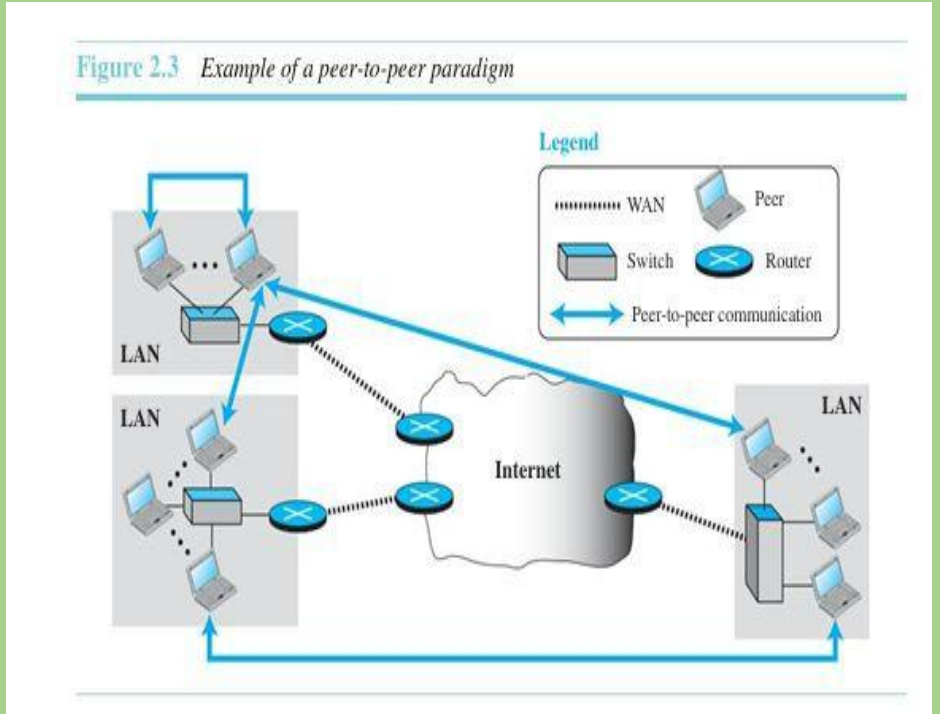
Examples of Client–Server Applications

- **WWW (World Wide Web)**
- **HTTP (HyperText Transfer Protocol)**
- **FTP (File Transfer Protocol)**
- **SSH (Secure Shell)**
- **E-mail Services**

New Paradigm: Peer-to-Peer (P2P)

The **Peer-to-Peer (P2P) Paradigm** is a communication model in which every computer (peer) can act as both a **service provider** and a **service requester**. There is no need for a dedicated server running all the time.

1. No dedicated server is required.
2. Responsibility is shared among all peers.
3. A peer can provide services to other peers.
4. A peer can receive services from other peers.
5. A peer can provide and receive services simultaneously.
6. Each computer has equal status in the network.
7. The workload is distributed among multiple peers.



Working of P2P Model

1. A peer requests a service from another peer.
2. The receiving peer provides the requested service.
3. The roles can change at any time.
4. The same peer can act as both client and server.

Advantages

1. No need for expensive dedicated servers.
2. Cost-effective.
3. Easily scalable.
4. Workload is distributed among peers.
5. Better utilization of network resources.

Disadvantages

1. Security is more challenging.
2. Difficult to manage distributed services.
3. Data availability depends on participating peers.
4. Not suitable for all Internet applications.

Applications of P2P

- Internet Telephony
- File Sharing
- BitTorrent
- IPTV
- Voice and Video Communication

Example

Internet Telephony

- Both users communicate directly.
- No dedicated server is required to continuously provide the service.
- Each participant acts as a peer.

File Sharing

- Users share files directly with each other.
- A user does not need to maintain a permanent server.

Mixed Paradigm

The **Mixed Paradigm** combines the features of both the **Client–Server** and **Peer-to-Peer (P2P)** paradigms to take advantage of the strengths of each model.

1. Combines **Client–Server** and **Peer-to-Peer** communication.
2. A central server is used only for specific tasks such as locating peers.
3. The actual service or data transfer occurs directly between peers.
4. Reduces the load on the server.
5. Improves scalability and efficiency.
6. Utilizes the advantages of both paradigms.

Working of Mixed Paradigm

1. A client contacts a server to find the address of a peer.
2. The server provides the peer's address.
3. The client establishes a direct connection with the peer.
4. Data or services are exchanged directly between peers.
5. The server is no longer involved in the actual communication.

Advantages

1. Reduces server workload.
2. More scalable than pure client-server architecture.
3. Cost-effective.
4. Efficient resource sharing.
5. Combines centralized control with distributed communication.

Applications

- Peer-to-Peer File Sharing
- Internet Telephony
- Skype
- BitTorrent
- IPTV

Client–Server Paradigm

The **Client–Server Paradigm** is a communication model in which a **client** sends a request for a service, and a **server** waits for requests, processes them, and sends back a response.

1. **Client**
   - Initiates communication.
   - Requests services from the server.
   - Runs only when needed.
2. **Server**
   - Provides services to clients.
   - Runs continuously and waits for requests.
   - Can serve multiple clients.
3. The **server's lifetime is infinite**, while the **client's lifetime is temporary**.

Application Programming Interface (API)

An **API (Application Programming Interface)** is a set of instructions that allows application programs to communicate with the operating system and the TCP/IP protocol suite.

Functions of API

- Open a connection
- Send data
- Receive data
- Close a connection

Common APIs

- Socket Interface
- Transport Layer Interface (TLI)
- STREAM

Socket

A Socket is an abstraction that acts as an endpoint for communication between a client process and a server process.

1. A socket is created by an application program.
2. Communication occurs between two sockets.
3. The client sends requests through its socket.
4. The server sends responses through its socket.
5. Sockets act as both source and destination of data.

Socket Address

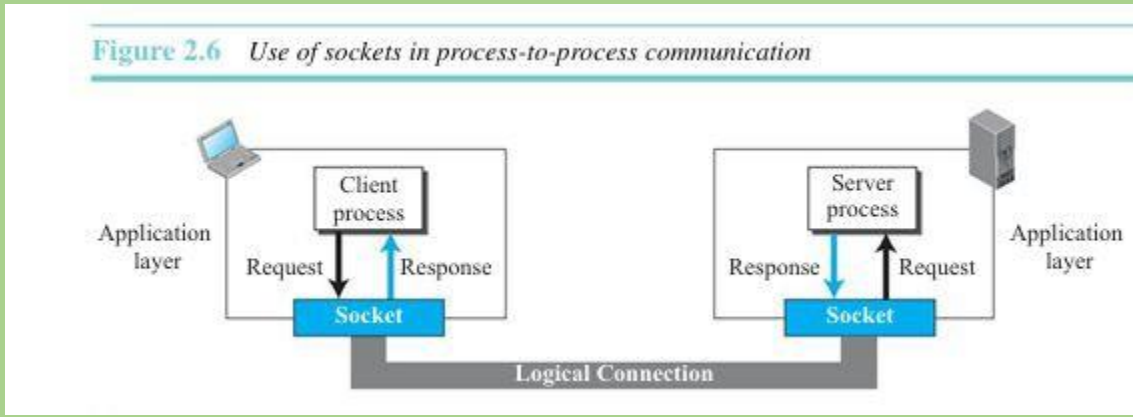
A **Socket Address** uniquely identifies a process on a computer.

Socket Address Formula

Socket Address = IP Address + Port Number

Components

1. **IP Address (32 bits)**
   - Identifies the computer.
2. **Port Number (16 bits)**
   - Identifies the specific application process.



Finding Socket Addresses

Server Side

Local Socket Address

- IP address is known by the operating system.
- Port number is fixed and assigned to the server.
- Example: HTTP uses Port 80.

Remote Socket Address

- Obtained from the client's request.
- Changes for each client connection.

Client Side

Local Socket Address

- IP address is assigned by the operating system.
- A temporary (**ephemeral**) port number is assigned for communication.

Remote Socket Address

- Consists of the server's IP address and port number.
- Can be obtained directly or through DNS.

Using Services of the Transport Layer

1. Application processes use the services provided by the **Transport Layer** for communication.
2. The TCP/IP protocol suite provides three common transport-layer protocols:
3. The choice of transport protocol affects the performance and capabilities of an application.
    - **UDP (User Datagram Protocol)**
    - **TCP (Transmission Control Protocol)**
    - **SCTP (Stream Control Transmission Protocol)**

UDP (User Datagram Protocol)

Characteristics

- Connectionless protocol
- Unreliable communication
- Message-oriented
- Fast and simple
- No flow control or congestion control

Advantages

- Low overhead
- Faster transmission
- Suitable for small messages

Applications

- Multimedia streaming
- Network management
- VoIP applications

Example

- Similar to sending letters through the post office.
- Each message is independent.

TCP (Transmission Control Protocol)

Characteristics

- Connection-oriented protocol
- Reliable communication
- Byte-stream oriented
- Provides flow control
- Provides error control
- Provides congestion control

Advantages

- Reliable data delivery
- Lost or corrupted data can be retransmitted
- Suitable for large data transfers

Applications

- Web browsing (HTTP/HTTPS)
- File Transfer (FTP)
- E-mail services

Example

- Similar to a telephone conversation where communication is established before exchanging information.

SCTP (Stream Control Transmission Protocol)

Characteristics

- Connection-oriented protocol
- Reliable communication
- Message-oriented
- Supports multistream communication
- Can maintain communication even if one path fails

Advantages

- Combines features of TCP and UDP
- Reliable and message-oriented
- Better fault tolerance

Applications

- Multimedia communication
- Telecommunication systems
- Applications requiring high reliability

STANDARD CLIENT-SERVER APPLICATIONS

During the evolution of the Internet, several **client–server applications** have been developed to provide different services. Understanding these applications helps in designing customized network applications. The major applications include:

- **HTTP/WWW** – Accessing web pages and websites.
- **FTP** – Transferring files between computers.
- **E-mail** – Sending and receiving electronic messages.
- **TELNET** – Remote login to another computer.
- **SSH** – Secure remote access and login.
- **DNS** – Mapping domain names to IP addresses.

World Wide Web (WWW)

- The **World Wide Web (WWW)** was proposed by Tim Berners-Lee in **1989** at CERN to facilitate the sharing of research information among scientists.
- The commercial use of the Web began in the **early 1990s**.
- The Web is a **global repository of information** where documents, called **web pages**, are stored on servers located around the world.
- The popularity of the Web is due to two important characteristics:
  - **Distributed Nature:** Web pages are stored on different servers worldwide, allowing unlimited growth.
  - **Linked Structure:** Web pages can be connected through hyperlinks, enabling easy navigation between related documents.
- The concept of linking documents originated from **Hypertext**, which allows users to move from one document to another by clicking a link.
- Today, the term **Hypermedia** is used because web pages can contain:
  - Text
  - Images
  - Audio
  - Video
  - Animations

Architecture of the World Wide Web (WWW)

The **World Wide Web (WWW)** is a **distributed client-server system** that allows users to access information over the Internet.

1. **Client**
   - The client is the user's computer or device.
   - It uses a **web browser** (such as Chrome, Firefox, or Edge) to request and view web pages.
2. **Server**
   - A server stores web pages and provides them to clients upon request.
3. **Website (Site)**
   - Information on the Web is distributed across many locations called **sites**.
   - Each site contains one or more web pages.
4. **Web Pages**
   - A web page is a file with a unique **name and address (URL)**.
   - Web pages can be:
     - **Simple Web Page:** Contains only information and no links to other pages.
     - **Composite Web Page:** Contains one or more links (hyperlinks) to other web pages within the same site or different sites.
5. **Hyperlinks**
   - Hyperlinks connect web pages together.
   - They allow users to navigate from one page to another.

Working of WWW Architecture

1. The user enters a URL in a web browser.
2. The browser (client) sends a request to the web server.
3. The server locates the requested web page.
4. The server sends the page back to the browser.
5. The browser displays the page, including any hyperlinks to other pages.

Retrieval of Web Documents, Web Client, and Web Server

Retrieval of Web Documents

A web document may contain references (hyperlinks) to other files such as text documents, images, videos, or web pages.

Example

Consider a scientific document:

- **File A** – Main document stored in Site I.
- **File B** – Large image stored in Site I.
- **File C** – Referenced text file stored in Site II.

Transactions Required

To view the complete document, **three separate transactions** are needed:

1. **First Transaction**
   - Retrieves the main document (File A).
   - File A contains references (links) to File B and File C.
2. **Second Transaction**
   - Triggered when the user clicks the image reference.
   - Retrieves the image file (File B).
3. **Third Transaction**
   - Triggered when the user clicks the text-file reference.
   - Retrieves the text file (File C).

- File A, File B, and File C are **independent web pages/files**.
- Each file has its own **name and address (URL)**.
- Even if File B and File C are referenced in File A, they can be accessed independently.
- Retrieving one file does not automatically retrieve the others.

Web Client (Browser)

A **Web Client** or **Browser** is software used to access and display web pages.

Architecture of a Browser

A browser consists of three main components:

1. Controller

- Receives input from the user through the keyboard or mouse.
- Controls access to web documents.

2. Client Protocols

- Used to communicate with web servers.
- Examples:
  - HTTP (HyperText Transfer Protocol)
  - FTP (File Transfer Protocol)

3. Interpreters

- Interpret and display the contents of web pages.
- Examples:
  - HTML Interpreter
  - Java Interpreter
  - JavaScript Interpreter

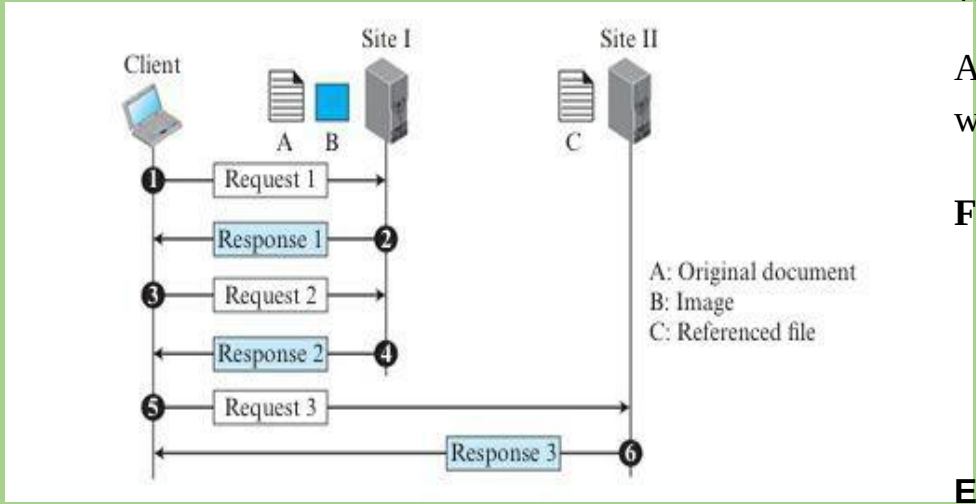
Working of a Browser

1. User enters a URL or clicks a link.
2. Controller sends a request using a client protocol.
3. Requested document is retrieved.
4. Interpreter processes the document.
5. Document is displayed on the screen.

Examples of Browsers

- Internet Explorer
- Netscape Navigator
- Firefox
- Google Chrome

Web Server



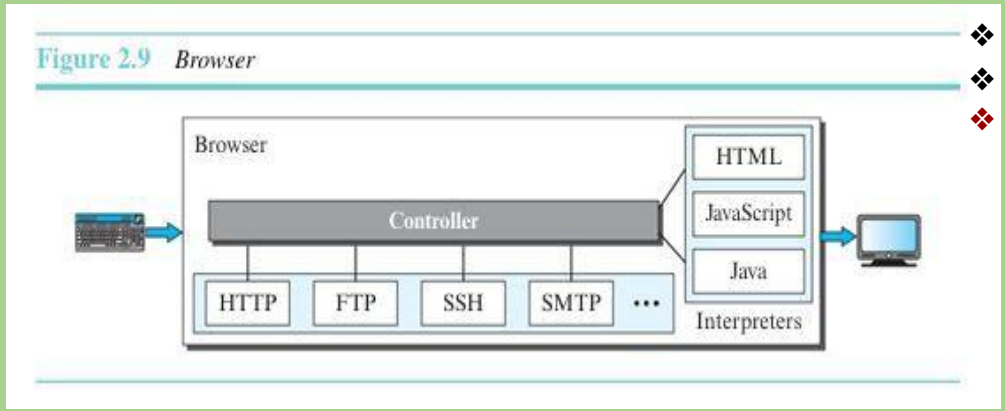
A **Web Server** stores web pages and delivers them to clients when requested.

Functions of a Web Server

- Stores web pages and files.
- Receives requests from browsers.
- Sends requested documents to clients.

Examples of Web Servers

- Apache Server
- Microsoft Internet Information Server (IIS)
- *A web document may contain references to other files, requiring separate transactions to retrieve each file. All referenced files are independent and have unique addresses. A web browser consists of a controller, client protocols, and interpreters that retrieve and display web pages. A web server stores web pages, responds to client requests, uses caching for faster access, and employs multithreading or multiprocessing to handle multiple users efficiently.*



Uniform Resource Locator (URL)

A **Uniform Resource Locator (URL)** is the unique address used to identify and locate a web page or resource on the Internet.To access a web page, four pieces of information are required:

1. **Protocol**
2. **Host**
3. **Port**
4. **Path**

Components of a URL

1. Protocol

- Specifies the client-server application used to access the resource.
- Indicates how data is transferred between the browser and the server.

Examples:

- HTTP (HyperText Transfer Protocol)
- HTTPS (HyperText Transfer Protocol Secure)
- FTP (File Transfer Protocol)

Example:http://

2. Host

- Identifies the server where the resource is stored.
- Can be:
  - An IP address
  - A Domain Name

Examples:

64.23.56.17
forouzan.com

3. Port

- A **16-bit integer** that identifies a specific service on the server.
- Usually predefined for common protocols.

Examples:

- HTTP → Port 80
- HTTPS → Port 443
- FTP → Port 21

If the default port is used, it is usually omitted from the URL.

Example::80

4. Path

- Specifies the location and name of the file on the server.
- Consists of directories followed by the file name.

UNIX Example:

/top/next/last/myfile

Here:

- top → top-level directory
- next → subdirectory
- last → subdirectory
- myfile → file name

URL Format

Without Port Number (Most Common)

protocol://host/path

Example:

http://www.forouzan.com/books/index.html

With Port Number

protocol://host:port/path

Example:

http://www.forouzan.com:80/books/index.html

- URL stands for **Uniform Resource Locator**.
- It uniquely identifies a web page or resource on the Internet.
- A URL consists of **Protocol, Host, Port, and Path**.
- The **host** identifies the server.
- The **port** identifies the service running on the server.
- The **path** identifies the exact file or resource.
- The **protocol** specifies how the resource is accessed.

Web Documents

Web documents are classified into three types:

1. Static Documents

- Fixed content documents stored on a web server.
- Users can only view a copy of the document.
- Content remains the same unless updated by the server administrator.
- Created using languages such as **HTML, XML, XSL, and XHTML**.

Example: A college website's information page.

2. Dynamic Documents

- Created by the server when requested by a browser.
- Content may change for each request.
- Generated using programs or scripts such as **JSP, ASP, and ColdFusion**.

Example: Current date and time displayed on a webpage.

3. Active Documents

- Contain programs or scripts that run on the client (browser) side.
- Used for animations, interactive forms, and user interactions.
- Created using **Java Applets** or **JavaScript**.

Example: Interactive games or animated graphics on a webpage.

HyperText Transfer Protocol (HTTP)

Definition

HTTP (HyperText Transfer Protocol) is the protocol used to transfer web pages between a web browser (client) and a web server.

- The **client** sends an HTTP request.
- The **server** sends an HTTP response.
- HTTP uses **TCP** as its transport protocol.
- **Server Port Number:** 80
- **Client Port Number:** Temporary (ephemeral) port.

Features of HTTP

- Client-Server protocol.
- Uses TCP for reliable communication.
- Requires a TCP connection before data transfer.
- Connection is terminated after communication.

Non Persistent and Persistent Connections

When a web page contains multiple objects (images, videos, etc.), HTTP can retrieve them using:

1. Non Persistent Connection

- A separate TCP connection is created for each request/response.
- Used in **HTTP/1.0**.

Steps

1. Client opens a TCP connection and sends a request.
2. Server sends the response and closes the connection.
3. Client receives the data and closes the connection.

Example

If a webpage contains **N images**, then:

- 1 connection for the HTML page
- N connections for N images

Total Connections = N + 1

Disadvantages

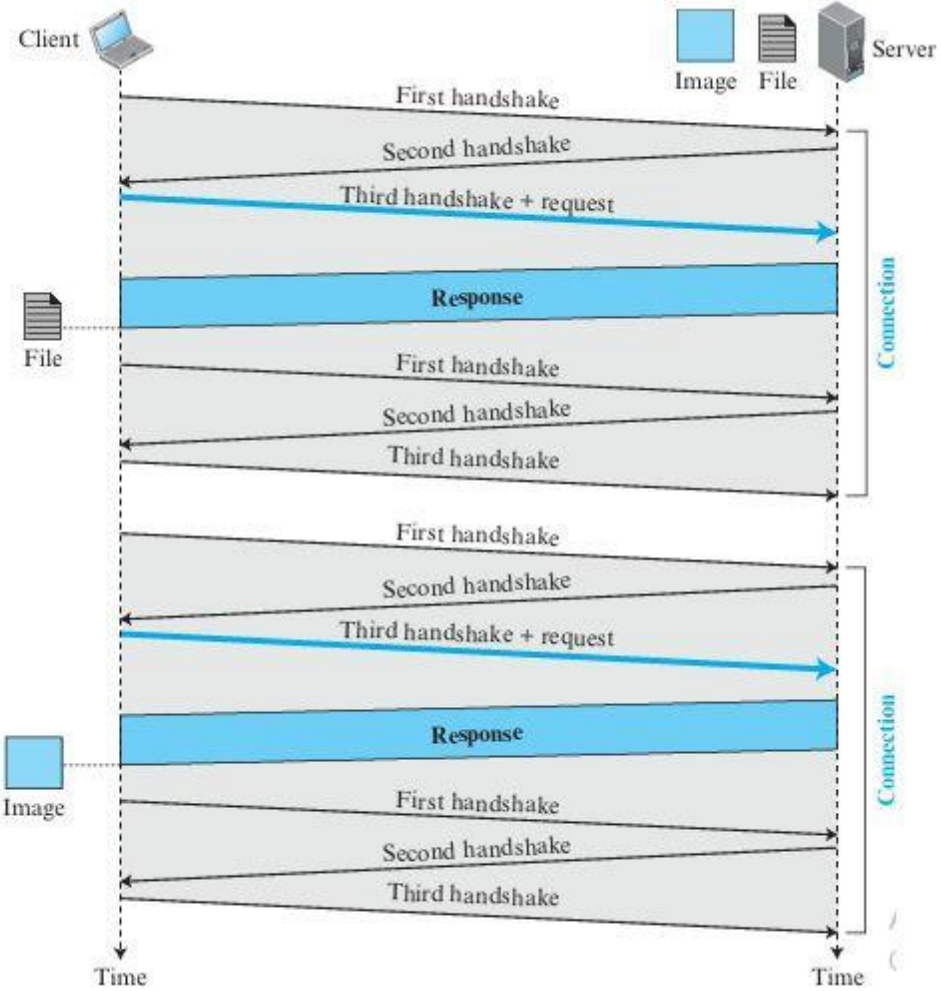
- High overhead.
- More connection setup and termination time.
- Increased server workload.
- Requires more buffers and resources.

2. Persistent Connection

- A single TCP connection is used for multiple requests and responses.
- Default in **HTTP/1.1**.

Advantages

- Reduced overhead.
- Faster page loading.
- Less network traffic.
- Better server performance.

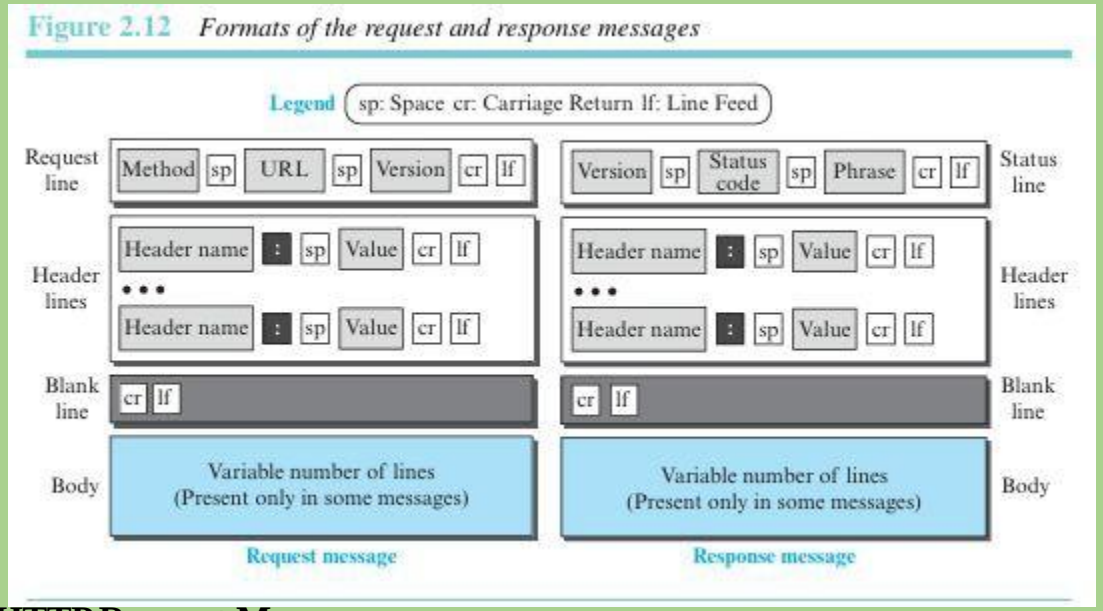


Nonpersistent Connection (Example 2.4)

- A webpage contains a text file with one linked image.
- Both files are stored on the same server.
- **Two separate TCP connections** are required:
  - One for the text file.
  - One for the image file.
- Each connection requires:
  - Connection establishment (TCP handshake).
  - Data transfer.
  - Connection termination.
- Multiple objects require multiple connections.
- Increased **round-trip time (RTT)** due to repeated handshakes.
- Additional buffers and variables must be allocated for each connection.
- Creates higher overhead, especially on the server.

Persistent Connection

- Default connection type in **HTTP/1.1**.
- A single TCP connection is used for multiple requests and responses.
- The server keeps the connection open after sending a response.
- The connection is closed:
  - At the client's request, or
  - When a timeout occurs.
- The sender usually specifies the length of the data being sent.
- For dynamically generated documents, if the length is unknown, the server closes the connection after transmission to indicate the end of data.
- Saves time and network resources.
- Requires only one set of buffers and variables at each site.
- Eliminates repeated connection establishment and termination.
- Reduces RTT and improves overall performance.



HTTP Request Message

An HTTP request message consists of four parts:

1. **Request Line**
2. **Header Lines**
3. **Blank Line**
4. **Body (Optional)**

1. Request Line

The request line contains three fields separated by a space (SP) and terminated by CRLF (Carriage Return and Line Feed).

Method

Specifies the type of request sent by the client.

Table 2.1 *Methods*

Method	*Action*
GET	Requests a document from the server
HEAD	Requests information about a document but not the document itself
PUT	Sends a document from the client to the server
POST	Sends some information from the client to the server
TRACE	Echoes the incoming request
DELETE	Removes the web page
CONNECT	Reserved
OPTIONS	Inquires about available options

URL

- Specifies the address and name of the requested web page.
- Identifies the resource to be accessed.

Example:

/index.html

Version

- Specifies the HTTP protocol version.
- Current version: **HTTP/1.1**

Example:

HTTP/1.1

2. Request Header Lines

- Zero or more header lines may follow the request line.
- Provide additional information from the client to the server.
- Allow the client to specify preferences and conditions.

Format

Header-Name: Value

Examples

Host: www.example.com
Accept: text/html
User-Agent: Chrome
If-Modified-Since: Mon, 10 Jun 2025

3. Blank Line

- A blank line separates the header section from the body section.
- Indicates the end of the headers.

4. Body (Optional)

- Present only in some request messages.
- Commonly used with **PUT** and **POST** methods.
- Contains:
    - Comments
    - Form data
    - Files to be uploaded
    - Information used to modify a web page

HTTP Response Message

An HTTP response message is sent by the **server to the client**. It consists of four parts:

1. **Status Line**
2. **Header Lines**
3. **Blank Line**
4. **Body (Optional)**

1. Status Line

The first line of a response message is called the **Status Line**.

Format

Version SP Status-Code SP Status-Phrase CRLF

Fields

Version

- Specifies the HTTP protocol version used by the server.
- Current version: **HTTP/1.1**

Example:

HTTP/1.1

Status Code Range	Meaning
100 – 199	Informational messages
200 – 299	Successful request
300 – 399	Redirection to another URL
400 – 499	Client-side error
500 – 599	Server-side error

Code	Meaning
200	OK
301	Moved Permanently
302	Found (Temporary Redirect)
404	Not Found
500	Internal Server Error

Status Phrase

- Text explanation of the status code.

Examples:

200 OK
404 Not Found
500 Internal Server Error

2. Response Header Lines

- Zero or more header lines may follow the status line.
- Provide additional information from the server to the client.
- Describe the document or server.

Format

Header-Name: Value

3. Blank Line

- Separates the header section from the body section.
- Marks the end of the header lines.

4. Body (Optional)

- Contains the actual document or data requested by the client.
- May include:
  - HTML pages
  - Images
  - Audio files
  - Video files
  - Error messages

FILE TRANSFER PROTOCOL (FTP)

File Transfer Protocol (FTP) is the standard TCP/IP protocol used for transferring files from one host to another. FTP solves problems caused by different file naming conventions, data representations, and directory structures. It is preferred over HTTP for transferring large files and files of different formats.

FTP Architecture

FTP uses a client-server model with two separate TCP connections.

Client Components

- User Interface
- Client Control Process
- Client Data Transfer Process

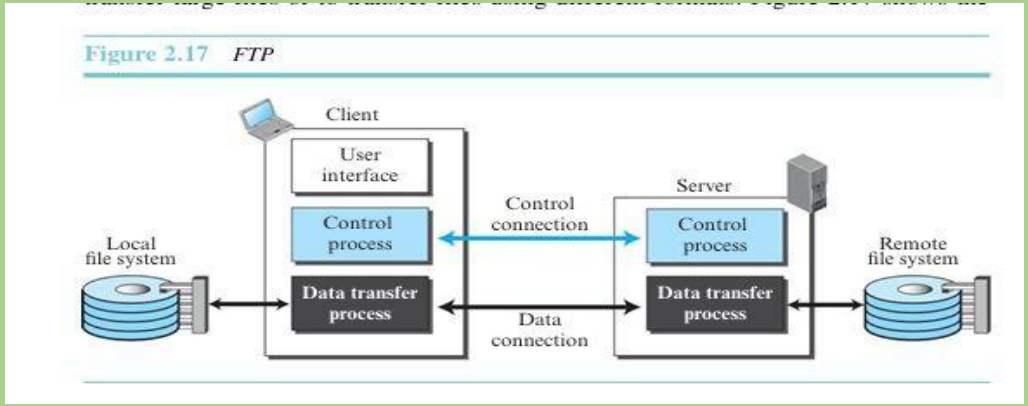
Server Components

- Server Control Process
- Server Data Transfer Process

Connections Used

1. **Control Connection** – Transfers commands and responses.
2. **Data Connection** – Transfers actual files.

Advantage: Separation of control information and data transfer makes FTP more efficient.



Lifetimes of FTP Connections

Control Connection

- Remains open throughout the FTP session.
- Used for sending commands and receiving responses.

Data Connection

- Opened only when a file transfer is required.
- Closed after the transfer is completed.
- Can be opened and closed multiple times during a single FTP session.

FTP Port Numbers

- Port 21 – Control Connection
- Port 20 – Data Connection

Control Connection

- Uses NVT ASCII character set.
- Communication takes place through commands and responses.
- Each command or response ends with CRLF (Carriage Return and Line Feed).

Common FTP Commands

Command	Function
USER	Sends username
PASS	Sends password
LIST	Lists directory contents
RETR	Retrieves a file
STOR	Stores a file
DELE	Deletes a file
QUIT	Ends FTP session
PORT	Specifies client port for data transfer

Establishment of Data Connection

1. The client performs a passive open using an ephemeral port.
2. The client sends the port number to the server using the PORT command.
3. The server performs an active open using port 20 and the client's ephemeral port.

Communication over Data Connection

Before file transfer, FTP defines:

- Data Structure
- File Type
- Transmission Mode

Data Structure

1. **File Structure** (Default) – Continuous stream of bytes.
2. **Record Structure** – File divided into records.
3. **Page Structure** – File divided into pages with page numbers.

File Types

1. ASCII File
2. EBCDIC File
3. Image (Binary) File

Transmission Modes

1. **Stream Mode** (Default) – Continuous stream of bytes.
2. **Block Mode** – Data transferred in blocks with headers.
3. **Compressed Mode** – Data transferred in compressed form.

FTP Error Codes

Code	Meaning
425	Cannot open data connection
450	File action not taken; file not available
452	Insufficient storage
500	Syntax error; unrecognized command
501	Syntax error in parameters or arguments
530	User not logged in

File Transfer Operations

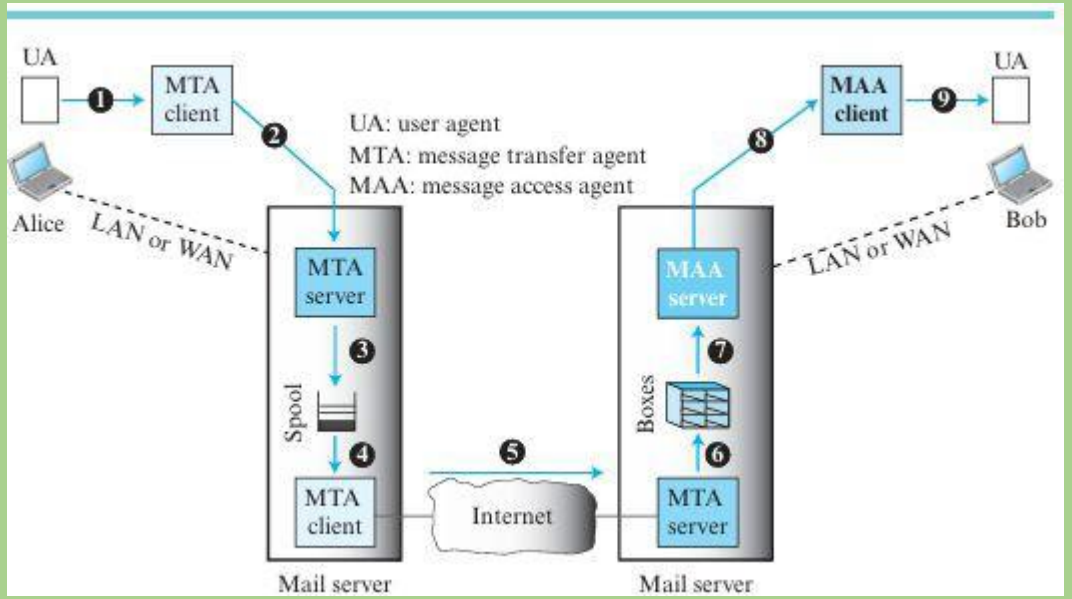
FTP supports three basic operations:

1. **Retrieve (Server → Client)**
   Transfers a file from the server to the client.
2. **Store (Client → Server)**
   Transfers a file from the client to the server.
3. **Directory Listing (Server → Client)**
   Sends directory information from the server to the client.

Electronic Mail (E-Mail)

- Electronic Mail (E-Mail) is an application-layer service that allows users to exchange messages over a computer network or the Internet.
- Unlike applications such as HTTP and FTP, where a server continuously waits for requests from clients and provides an immediate response, e-mail works differently.
- E-mail is considered a **one-way transaction**, meaning that when one user sends a message to another, a response is not mandatory.
- The receiver may choose to reply or may not respond at all. If a reply is sent, it is treated as a separate one-way transaction.
- In an e-mail system, it is neither practical nor logical for every user to run a server program continuously and wait for incoming messages.
- Users may switch off their computers or disconnect from the network when they are not using them.
- To overcome this limitation, e-mail communication relies on **intermediate mail servers**.
- These servers store, forward, and deliver messages between users. The sender's e-mail client sends the message to a mail server, which then transfers it to the recipient's mail server.
- When the recipient becomes available, they can access and retrieve the message using their e-mail client.

- Thus, e-mail implements the client-server model in an indirect manner.
- Users run only client programs whenever they need to send or receive messages, while dedicated mail servers handle the storage and delivery of e-mails.
- This approach allows reliable communication even when the sender and receiver are not online at the same time.



Architecture of E-Mail

- The architecture of an e-mail system consists of users, mail servers, mailboxes, queues, and different software agents that work together to send and receive messages.
- In a typical e-mail scenario, the sender (Alice) and the receiver (Bob) are connected to their respective mail servers through a LAN or WAN.
- Each user has a **mailbox** on the mail server where received messages are stored, and a **queue (spool)** where outgoing messages wait before being sent.
- When Alice wants to send an e-mail, she uses a **User Agent (UA)** to compose and submit the message to her mail server.
- The message is then placed in the queue and transferred across the Internet using a **Mail Transfer Agent (MTA)**. The MTA follows the client-server model, where the client transfers the message to the destination mail server, and the server runs continuously to accept incoming messages.
- Once the message reaches Bob's mail server, it is stored in Bob's mailbox. To read the message, Bob uses a **Message Access Agent (MAA)**, which retrieves the message from the mail server. Unlike the MTA, which is a **push protocol** that pushes messages to the destination server, the MAA is a **pull protocol** that allows users to retrieve messages when needed. This architecture eliminates the need for users to keep their computers running continuously and enables reliable e-mail communication even when the sender and receiver are not online at the same time.

User Agent (UA)

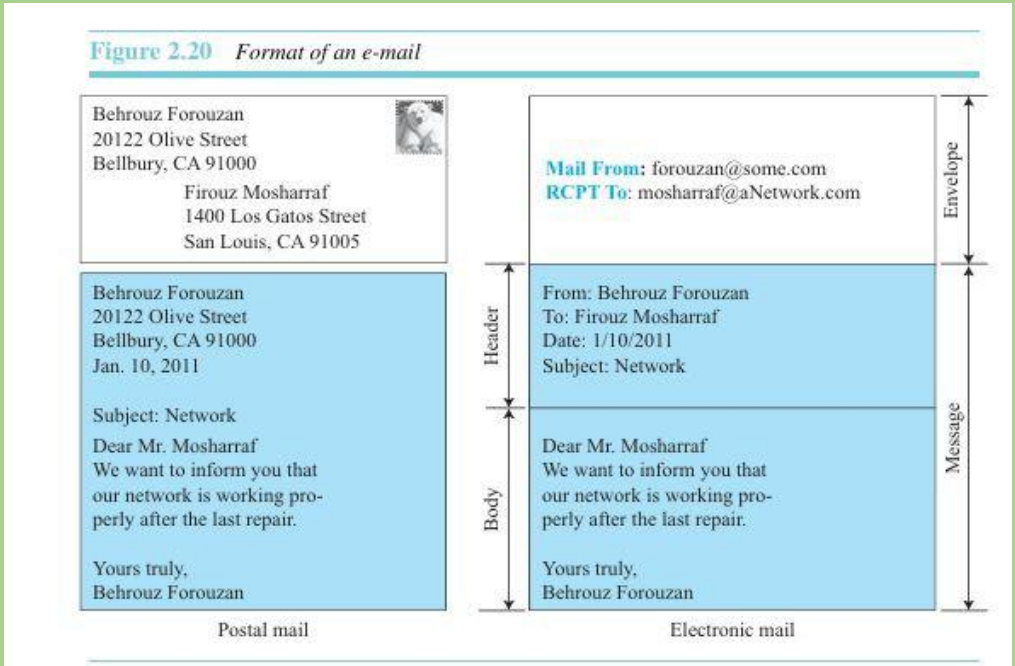
- A **User Agent (UA)** is the first component of an electronic mail system that provides services to users for creating, sending, receiving, reading, replying to, and forwarding e-mail messages.
- It is a software program that acts as an interface between the user and the e-mail system and also manages local mailboxes on the user's computer.
- User agents are of two types: **command-driven** and **GUI-based**. Command-driven user agents were commonly used in the early days of e-mail and require users to enter simple keyboard commands to perform tasks. Examples include *mail*, *pine*, and *elm*. Modern e-mail systems use **GUI-based user agents**, which provide graphical elements such as icons, menus, buttons, and windows, making e-mail operations easier and more user-friendly. Examples include Microsoft Outlook and Eudora.

Sending Mail

- To send an e-mail, the user creates the message using the User Agent. An e-mail is similar to postal mail and consists of an **envelope** and a **message**. The envelope contains information such as the sender's address, receiver's address, and other delivery details. The message itself contains two parts: a **header** and a **body**. The header includes information such as the sender, receiver, subject, and other control details, while the body contains the actual content of the message. Once the message is prepared, the User Agent sends it to the mail server for delivery.

Receiving Mail

- When receiving e-mail, the User Agent is activated either by the user or automatically through a timer. If new mail is available in the mailbox, the User Agent notifies the user. It then displays a list of received messages, showing summary information such as the sender's e-mail address, subject, and date or time of receipt. The user can select any message from the list and view its complete contents on the screen. Thus, the User Agent simplifies the process of accessing and managing received e-mails.

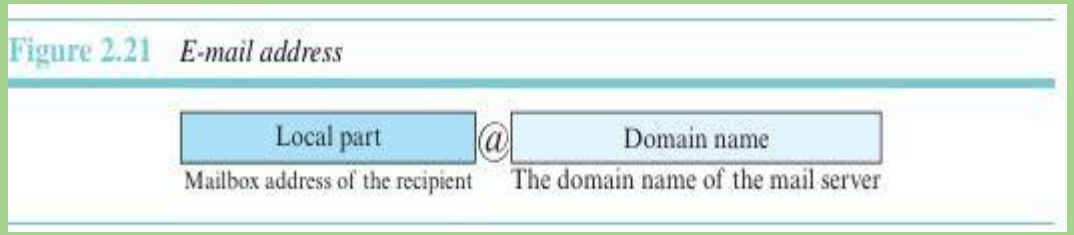


E-Mail Addresses

- To ensure proper delivery of e-mail messages, an e-mail system uses a unique addressing scheme. In the Internet, an e-mail address consists of two parts separated by the **@ (at) symbol**: the **local part** and the **domain name**.
- The local part identifies the user's mailbox, which is a special file where all incoming messages for that user are stored until they are retrieved.
- The domain name identifies the mail server or mail exchanger responsible for handling the user's e-mails. The domain name may be obtained from the Domain Name System (DNS) or may represent the name of an organization. This addressing system ensures that every e-mail message reaches the correct destination efficiently and accurately.

Mailing List (Group List)

- Electronic mail also supports **mailing lists**, also known as **group lists**, which allow a single name or alias to represent multiple e-mail addresses. Instead of sending the same message individually to several recipients, a user can send it to the mailing list alias. The e-mail system then checks the alias database and automatically expands the alias into all the e-mail addresses associated with it. Separate copies of the message are created for each member of the group and forwarded to the Mail Transfer Agent (MTA) for delivery. Mailing lists simplify communication with groups and make it easier to send information to multiple recipients simultaneously.



Message Transfer Agent (MTA) – SMTP

- The **Message Transfer Agent (MTA)** is responsible for transferring e-mail messages from the sender to the receiver through mail servers.
- In an e-mail system, three client-server interactions are involved: the first between the sender and the sender's mail server, the second between the sender's mail server and the receiver's mail server, and the third between the receiver and the receiver's mail server.
- The first two interactions use a Message Transfer Agent (MTA), while the third uses a Message Access Agent (MAA).
- The standard Internet protocol used by the MTA is the **Simple Mail Transfer Protocol (SMTP)**. SMTP is used twice during e-mail transmission: first, to transfer the message from the sender's user agent to the sender's mail server, and second, to transfer the message between the sender's and receiver's mail servers.
- A different protocol, such as POP or IMAP, is used later by the receiver to access the message.

- SMTP operates using a **command-response mechanism**.
- The MTA client sends commands to the MTA server, and the server replies with appropriate responses. Each command and response is terminated by a two-character end-of-line sequence consisting of a **carriage return (CR)** and a **line feed (LF)**.
- Common SMTP commands include **HELO** (to identify the client), **MAIL FROM** (to specify the sender's address), **RCPT TO** (to specify the recipient's address), **DATA** (to indicate the start of the message content), and **QUIT** (to terminate the session).
- Responses are sent from the server to the client in the form of **three-digit status codes**, often accompanied by textual information that describes the result of the command. Thus, SMTP provides a reliable and standardized method for transferring e-mail messages across the Internet.

Mail Transfer Phases in SMTP

- The transfer of an e-mail message using **Simple Mail Transfer Protocol (SMTP)** occurs in three phases: **Connection Establishment, Message Transfer, and Connection Termination**.
- During the **Connection Establishment** phase, the SMTP client first creates a TCP connection with the SMTP server using port 25. The server then sends a **220 (Service Ready)** response to indicate that it is ready to receive mail.
- If the service is unavailable, the server sends a **421** response. Next, the client sends a **HELO** command along with its domain name to identify itself to the server.
- The server acknowledges this request by sending a **250 (Request Completed)** response or another appropriate code.
- Once the connection is established, the **Message Transfer** phase begins.
- The client sends a **MAIL FROM** command containing the sender's e-mail address, and the server responds with a **250** code if the command is accepted.
- The client then sends a **RCPT TO** command specifying the recipient's e-mail address, and the server again responds appropriately. If there are multiple recipients, this step is repeated for each recipient. After specifying the recipients, the client sends the **DATA** command to indicate the start of the message content.
- The server responds with **354 (Start Mail Input)**, allowing the client to transmit the message.

- The actual message is then sent line by line, with each line ending in a carriage return and line feed (CRLF). The end of the message is indicated by a line containing only a single period (.). After receiving the complete message, the server sends a **250 (OK)** response to confirm successful receipt.
- Finally, in the **Connection Termination** phase, the client ends the SMTP session by sending the **QUIT** command. The server responds with a **221** code to indicate that the connection is being closed. The TCP connection is then terminated. These three phases ensure reliable and orderly transfer of e-mail messages between mail servers on the Internet.

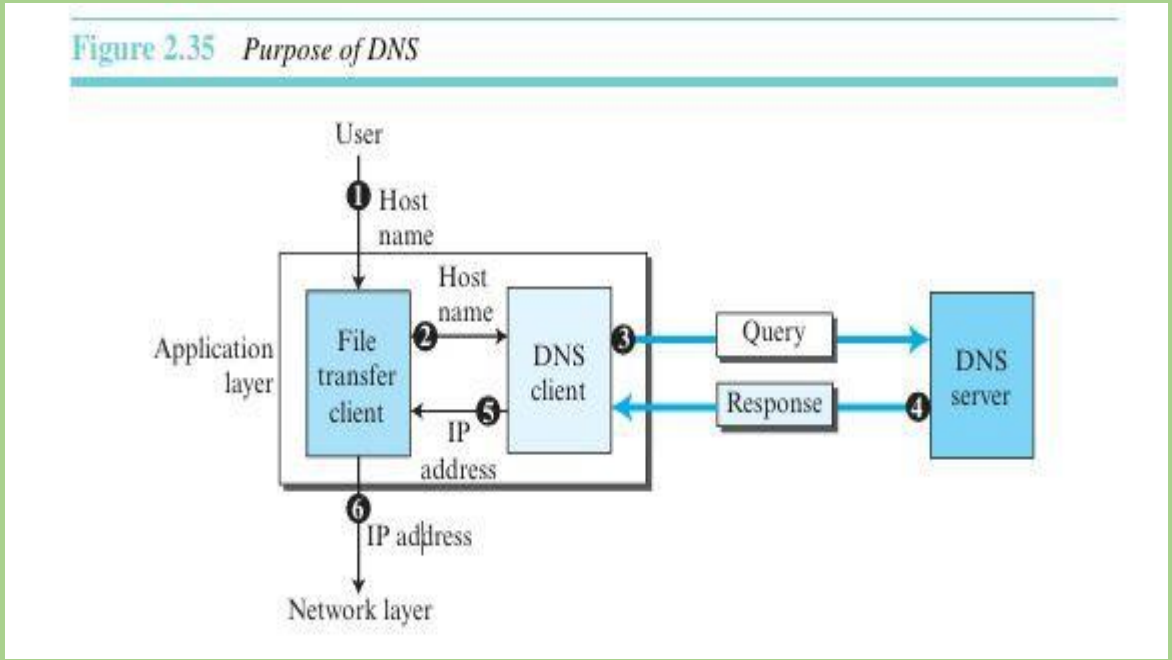
Domain Name System (DNS)

- The **Domain Name System (DNS)** is an important client-server application in the TCP/IP protocol suite that helps other Internet applications by translating human-readable domain names into IP addresses.
- Although computers and networking devices communicate using numerical IP addresses, people find it easier to remember names such as *google.com* or *filesource.com*.
- Therefore, DNS acts as a distributed directory service that maps domain names to their corresponding IP addresses. Similar to a telephone directory that maps people's names to telephone numbers, DNS maps host names to network addresses.
- Since the Internet contains billions of devices, maintaining a single centralized directory would be impractical and unreliable.
- Instead, DNS distributes its database across many DNS servers located around the world.
- This distributed approach improves efficiency, scalability, and reliability because users can access nearby DNS servers, and the failure of one server does not affect the entire system.

- When a user wants to access a remote service such as a file transfer server, the user typically knows only the server's domain name and not its IP address.
- The process begins when the user enters the host name into a client application such as an FTP client.
- The client passes the host name to the DNS client, which sends a query to a DNS server whose address is already known to the computer.
- The DNS server searches its database and returns the corresponding IP address of the requested host.
- The DNS client then provides this IP address to the application client, which uses it to establish a connection with the remote server.
- Thus, DNS serves as a vital Internet service that enables users to access resources using easy-to-remember names while allowing the TCP/IP protocols to operate using IP addresses.

DNS and Multiple Connections

- Before a client can connect to a file transfer server, the server's domain name must first be translated into an IP address using DNS. Therefore, at least **two connections** are required: the first between the **DNS client and DNS server** for name resolution, and the second between the **file transfer client and file transfer server** for the actual data transfer. In some cases, DNS may require additional connections to other DNS servers to find the correct IP address.



Name Space

- A **name space** is a system used to assign unique names to devices on the Internet so that each name corresponds to a unique IP address. Name spaces can be organized as **flat** or **hierarchical**. In a flat name space, each name is simply a sequence of characters without any structure, requiring central control to avoid duplication. This approach is not suitable for large networks like the Internet. A hierarchical name space, on the other hand, consists of multiple levels, allowing the responsibility of assigning names to be distributed among different organizations. This makes it scalable and ensures that names remain unique. For example, two organizations can use the same host name, such as *caesar*, but their complete domain names (*caesar.first.com* and *caesar.second.com*) remain unique.

Domain Name Space

- To implement a hierarchical naming system, the Internet uses a **Domain Name Space**, which is organized as an inverted tree structure with the **root** at the top. Each node in the tree has a **label**, which can contain up to 63 characters. The labels of sibling nodes must be different to ensure uniqueness. Every node also has a **domain name**, which is formed by a sequence of labels separated by dots (.). Domain names are read from the node toward the root. A complete domain name, called a **fully qualified domain name (FQDN)**, ends with the root label, represented by a trailing dot. This hierarchical structure allows DNS to efficiently organize and manage millions of domain names across the Internet.

Figure 2.36 *Domain name space*

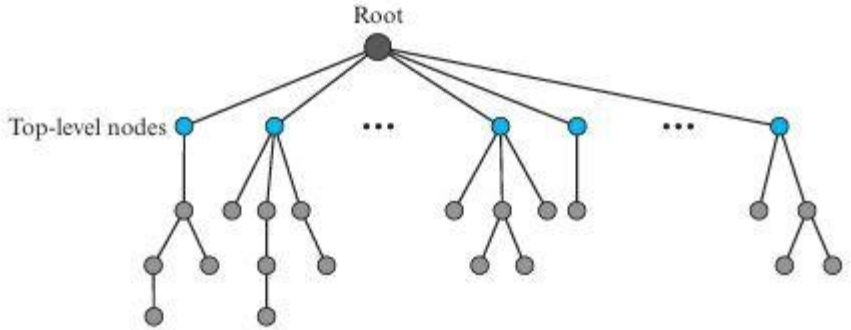


Figure 2.37 *Domain names and labels*

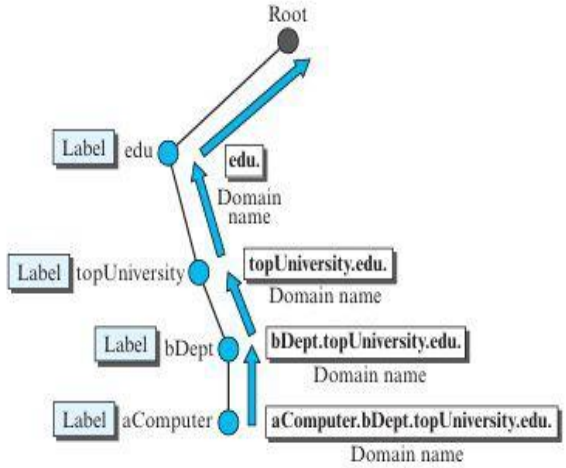
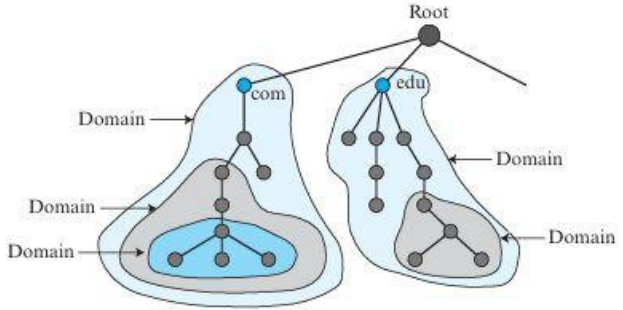


Figure 2.38 *Domains*



Distribution of Name Space

- The DNS name space is distributed among many **DNS servers** because storing all domain information on a single server would be inefficient and unreliable. To manage the large amount of data, the domain name space is divided into **domains** and **subdomains**, and different DNS servers are assigned responsibility for them. This creates a **hierarchy of name servers**, where each server is authoritative for a particular domain or subdomain.

Zone

- A **zone** is the portion of the DNS name space for which a DNS server is responsible. If a domain is not divided into subdomains, the **domain and zone are the same**. However, when a domain is divided and authority is delegated to lower-level servers, the original server retains only part of the information while the detailed information is maintained by the servers responsible for the subdomains. This distributed approach improves the efficiency, scalability, and reliability of the DNS system.

Figure 2.39 *Hierarchy of name servers*

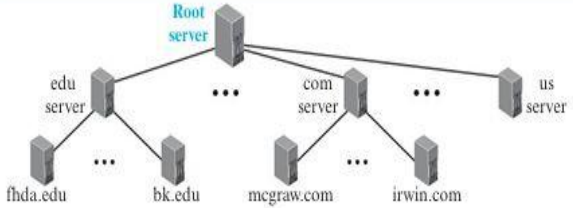
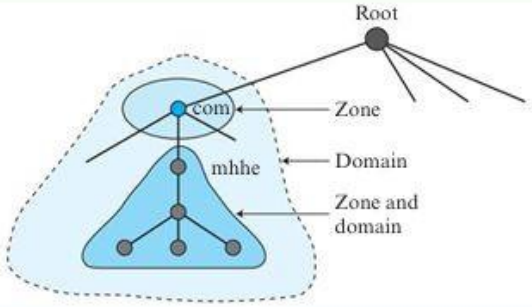


Figure 2.40 *Zone*



Root Server

- A **Root Server** is a DNS server whose zone covers the entire domain name space.
- Instead of storing detailed information about all domains, it delegates authority to lower-level DNS servers and maintains references to them. Multiple root servers are distributed across the world to improve reliability and availability of the DNS system.

Primary and Secondary Servers

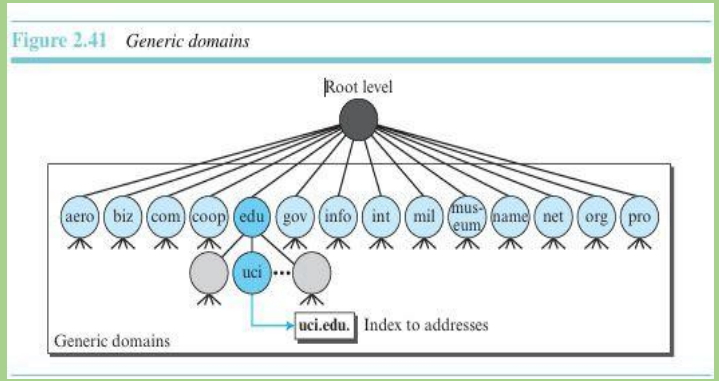
- DNS uses two types of authoritative servers: **Primary Servers** and **Secondary Servers**.
- A **Primary Server** stores the original zone file and is responsible for creating, maintaining, and updating it.
- A **Secondary Server** obtains a copy of the zone file from a primary or another secondary server and stores it locally. Secondary servers cannot modify the zone file; any updates must be made on the primary server and then transferred to the secondary servers.
- Both primary and secondary servers are authoritative for their zones. The purpose of secondary servers is to provide redundancy and improve reliability, ensuring that DNS services continue even if the primary server fails. A server can act as a primary server for one zone and a secondary server for another zone.

DNS in the Internet

- DNS is a protocol used to translate domain names into IP addresses on the Internet. Originally, the Internet domain name space was divided into **Generic Domains**, **Country Domains**, and **Inverse Domains**. However, inverse domains, which were used to find a hostname from an IP address, have been deprecated due to the rapid growth of the Internet. Therefore, modern DNS mainly focuses on generic and country domains.

Generic Domains

- **Generic Domains** classify hosts based on their general purpose or type of organization. Each domain in the DNS hierarchy acts as an index to the domain name space database. Common generic domains include **.com** for commercial organizations, **.edu** for educational institutions, **.org** for non-profit organizations, **.gov** for government agencies, and **.net** for network service providers. These domains help organize and identify different types of Internet resources efficiently.



Peer-to-Peer (P2P) Paradigm

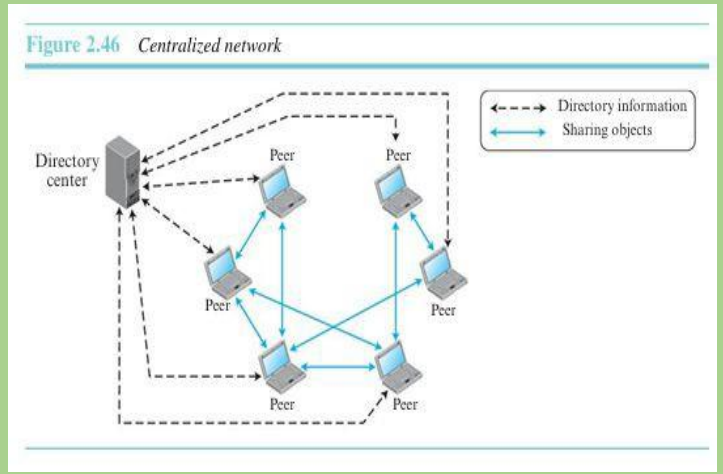
- The **Peer-to-Peer (P2P) paradigm** is a network model in which computers communicate directly with each other without relying on a central server. Each computer, called a **peer**, can act as both a client and a server. The concept of P2P file sharing began with **WWIVnet** in 1987 and later evolved through systems such as **Freenet** (1999), **Napster** (1999), **Gnutella** (2000), **Kazaa**, **BitTorrent**, **WinMX**, and **GNUnet** (2001). P2P networks became popular for file sharing because they enable decentralized communication and resource sharing among users.

P2P Networks

- A **Peer-to-Peer (P2P) network** is a distributed network in which Internet users share resources directly with one another. When a peer has a file, such as an audio, video, or document file, it makes the file available to other peers in the network. Any interested peer can connect directly to the computer storing the file and download it. After downloading, the peer can also share the same file with others, increasing the number of available copies in the network. As peers continuously join and leave the network, maintaining information about active peers and file locations becomes an important challenge. Based on how this information is managed, P2P networks are classified into **centralized** and **decentralized** networks.

Centralized P2P Networks

- A **Centralized Peer-to-Peer (P2P) Network** combines the client-server and peer-to-peer paradigms and is therefore often called a **hybrid P2P network**. In this model, a central server maintains a directory containing information about peers and the files they share, while the actual storage and transfer of files take place directly between peers. When a peer joins the network, it registers with the central server by providing its IP address and a list of files available for sharing. When another peer searches for a particular file, it sends a query to the central server. The server searches its directory and returns the IP addresses of peers that possess the requested file. The requesting peer then connects directly to one of those peers and downloads the file. The directory is continuously updated as peers join or leave the network.



- Although centralized networks simplify file searching and directory management, they have several disadvantages. The central server can become a bottleneck due to heavy traffic, reducing system performance. It is also vulnerable to attacks and failures; if the central server becomes unavailable, the entire network may stop functioning. **Napster** was a well-known example of a centralized P2P network. Its reliance on central servers contributed to legal and operational challenges, eventually leading to its shutdown in 2001.

Decentralized P2P Networks

- A **Decentralized Peer-to-Peer (P2P) Network** does not rely on a central server or directory system. Instead, peers organize themselves into an **overlay network**, which is a logical network built on top of the physical network. Based on how the peers are connected, decentralized networks are classified as **unstructured** or **structured**.
- In an **Unstructured P2P Network**, peers are connected randomly. To locate a file, a search query is flooded across the network, which generates high traffic and may not always find the desired file efficiently. Examples of unstructured P2P networks include **Gnutella** and **Freenet**.

- In a **Structured P2P Network**, peers are connected according to predefined rules, enabling faster and more efficient file searches. These networks commonly use a **Distributed Hash Table (DHT)** to organize and locate resources. DHT technology is widely used in systems such as distributed databases, content distribution systems, DNS, and P2P file-sharing applications. **BitTorrent** is a popular example of a structured P2P network that uses DHT for efficient resource discovery and file sharing.

Case Study: BitTorrent – A Popular Peer-to-Peer (P2P) Network

Introduction

BitTorrent, designed by Bram Cohen, is one of the most widely used Peer-to-Peer (P2P) file-sharing protocols. Unlike traditional file-sharing systems, where a single peer provides the entire file, BitTorrent divides a large file into smaller pieces called **chunks**. Peers in the network simultaneously download and upload these chunks, making file distribution faster and more efficient.

Working of BitTorrent

The group of peers participating in the sharing of a file is called a **swarm**. A peer that possesses the complete file is known as a **seed**, while a peer that has only some parts of the file and is still downloading the rest is called a **leech**. As peers download chunks, they also upload them to others, creating a collaborative sharing environment.

BitTorrent with a Tracker

In the original BitTorrent design, a central entity called a **tracker** manages the swarm. When a new peer wants to download a file, it first obtains a **torrent file (metafile)** containing information about the file pieces and the tracker's address. The peer then contacts the tracker, which provides a list of neighboring peers participating in the torrent. The new peer downloads missing chunks from these peers and uploads available chunks to others. Once all chunks are collected, the peer becomes a seed and can continue helping other users.

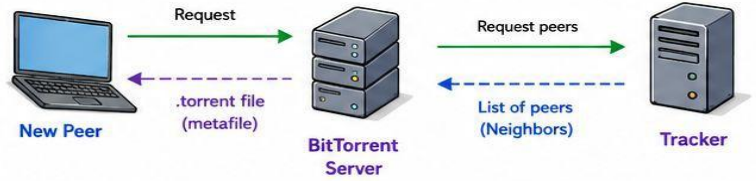
Fairness and Efficiency Mechanisms

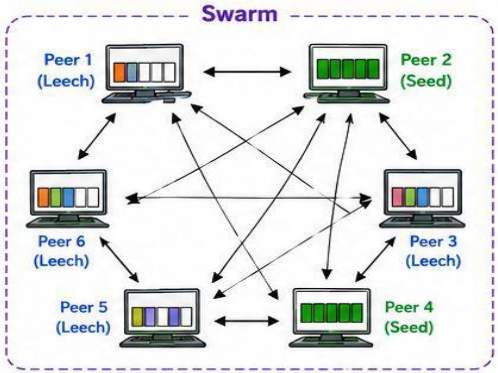
BitTorrent uses several strategies to ensure fairness and efficient resource utilization. Each peer maintains connections with only a limited number of neighbors, typically four, to avoid overloading the network. Peers are categorized as **unchoked** (active connections) or **choked** (inactive connections). Every few seconds, peers evaluate neighboring nodes and prefer those offering better data transfer rates. BitTorrent also uses **optimistic unchoking**, which periodically allows a new peer to receive data even if it has not yet contributed. Another important strategy is **rarest-first**, where peers prioritize downloading the least available pieces first, ensuring balanced distribution of file chunks throughout the swarm.

BitTorrent: How a P2P File Sharing Network Works

1 BitTorrent with a Tracker

1. A user gets the torrent file from the BitTorrent server.
2. The peer contacts the tracker to get a list of peers.
3. The peer connects to other peers (neighbors) and exchanges pieces.





Key Points

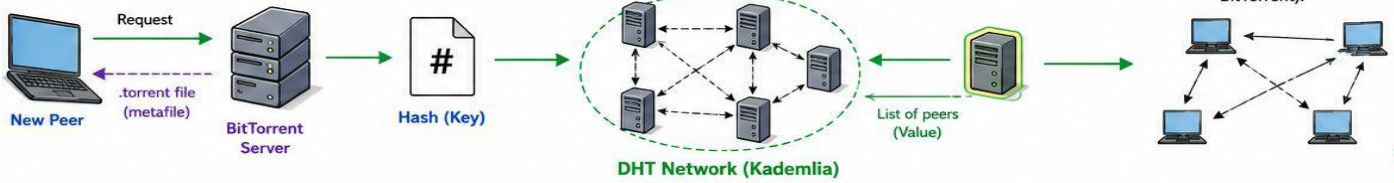
- File is divided into pieces (chunks).
- Peers download pieces from others and upload pieces they have.
- When a peer has all pieces, it becomes a Seed.
- The tracker helps new peers find others in the swarm.

Peer Connection Policy (for fairness and efficiency)

- **Tit-for-tat:** Upload to those who upload to you.
- **Choking / Unchoking:** Keep best performing peers unchoked.
- **Optimistic Unchoke:** Every 30 sec, unchoke one random peer.
- **Rarest-First:** Download rarest pieces first.

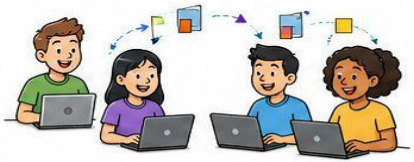
2 Trackerless BitTorrent using DHT (Kademlia)

1. A peer gets the torrent file (metafile).
2. The peer computes the hash of the metadata (as the key).
3. The peer sends the key into the DHT (Kademlia network).
4. The DHT finds the node responsible for the key and returns the list of peers (Value).
5. The peer gets the list of peers and starts exchanging pieces (as in regular BitTorrent).



Benefits of Trackerless (DHT)

- ✓ No single point of failure
- ✓ Decentralized and scalable
- ✓ New peers can still join even if some nodes go down
- ✓ More resilient and efficient



BitTorrent is a collaborative P2P protocol that divides a file into pieces, and peers download and upload pieces simultaneously. It achieves fast, efficient and scalable file sharing. **Modern versions use DHT to remove the need for a central tracker.**

Trackerless BitTorrent

One limitation of the original BitTorrent system was its dependence on the tracker. If the tracker failed, new peers could not join the swarm. To overcome this problem, modern BitTorrent implementations use a **Distributed Hash Table (DHT)**, particularly the **Kademlia DHT** algorithm. In this approach, the tracking function is distributed among multiple peers instead of relying on a central server. A joining peer sends a request to the DHT network, which locates the responsible node and returns a list of peers participating in the torrent. This eliminates the single point of failure and improves scalability and reliability.

Conclusion

BitTorrent is an efficient and scalable P2P file-sharing protocol that distributes the workload among participating peers. By dividing files into chunks and encouraging peers to both upload and download data, BitTorrent achieves faster transfers and better resource utilization. The introduction of trackerless operation using DHT further enhanced its reliability by removing dependence on a central tracker.

PART A – Short Answer Questions (2 Marks)

1. Define a computer network.
2. List the advantages of computer networks.
3. What are end devices? Give examples.
4. What are connecting devices in a network?
5. Define transmission media.
6. Differentiate between wired and wireless media.
7. What is a PAN?
8. What is a LAN?
9. What is a MAN?
10. What is a WAN?
11. Define an intranet.
12. What is an internetwork?
13. What is the role of a router?
14. What is the role of a switch?
1. Define Point-to-Point WAN.
2. Define Switched WAN.
3. Differentiate between Internet and Intranet.
4. What is circuit switching?
5. What is packet switching?
6. Define ISP (Internet Service Provider).
7. What is DSL?
8. What is the TCP/IP protocol suite?
9. Why does TCP/IP use layering?
10. List the layers of the TCP/IP model.
11. What is encapsulation?
12. What is decapsulation?
13. Define multiplexing.
14. Define demultiplexing.
15. What is the client-server paradigm?
16. What is the peer-to-peer paradigm?

Short Essay Questions (5 Marks)

1. Explain the need for computer networks.
2. Describe the components of a computer network.
3. Explain different types of transmission media.
4. Compare LAN, MAN, and WAN.
5. Explain the concept of an internetwork.
6. Discuss the role of routers and switches in networking.
7. Explain Point-to-Point WAN and Switched WAN.
8. Differentiate between Internet and Intranet.
9. Explain circuit switching with an example.
10. Explain packet switching with an example.
11. Compare circuit-switched and packet-switched networks.
12. Describe the structure of the Internet.
13. Explain the different types of ISPs.
14. Discuss various methods of accessing the Internet.
15. Explain the TCP/IP protocol suite.
16. Discuss the advantages of layering in TCP/IP.
17. Explain the functions of the TCP/IP layers.
18. Describe encapsulation and decapsulation.
19. Explain multiplexing and demultiplexing.
20. Compare Client–Server and Peer-to-Peer paradigms.

Long Answer Questions (10 Marks)

1. Explain the components of a computer network with suitable diagrams.
2. Discuss different types of computer networks (PAN, LAN, CAN, MAN, WAN, GAN).
3. Explain the concept of internetworking and the role of routers and switches.
4. Describe Point-to-Point WAN and Switched WAN with examples.
5. Compare circuit switching and packet switching in detail.
6. Explain the structure of the Internet and the role of backbone, provider, and customer networks.
7. Discuss the functions and types of Internet Service Providers (ISPs).
8. Explain various methods used to access the Internet.
9. Describe the TCP/IP protocol suite and its layered architecture.
10. Explain the responsibilities of each layer in the TCP/IP model.
11. Explain logical communication in the TCP/IP protocol suite.
12. Discuss encapsulation and decapsulation with neat diagrams.
13. Explain multiplexing and demultiplexing with suitable examples.
14. Describe the client-server paradigm and its applications.
15. Explain the peer-to-peer paradigm, its advantages, disadvantages, and applications.

Important University-Level Questions

1. Explain the layered architecture of the TCP/IP protocol suite.
2. Compare circuit switching and packet switching.
3. Explain the structure of the Internet.
4. Discuss the role of routers and switches in internetworking.
5. Explain encapsulation and decapsulation in TCP/IP.
6. Describe multiplexing and demultiplexing.
7. Compare client-server and peer-to-peer paradigms.
8. Explain different methods of Internet access.
9. Discuss the various types of computer networks.
10. Explain the components of a computer network.

Case Study Questions

Case Study 1: College Network

Case Study 2: Online Video Streaming

Case Study 3: Office Intranet